MOCCA 2025
Fundamentals of Character Animation and Motion Control

Lecture 04:

Keyframe Animation

Libin Liu

School of Intelligence Science and Technology
Peking University

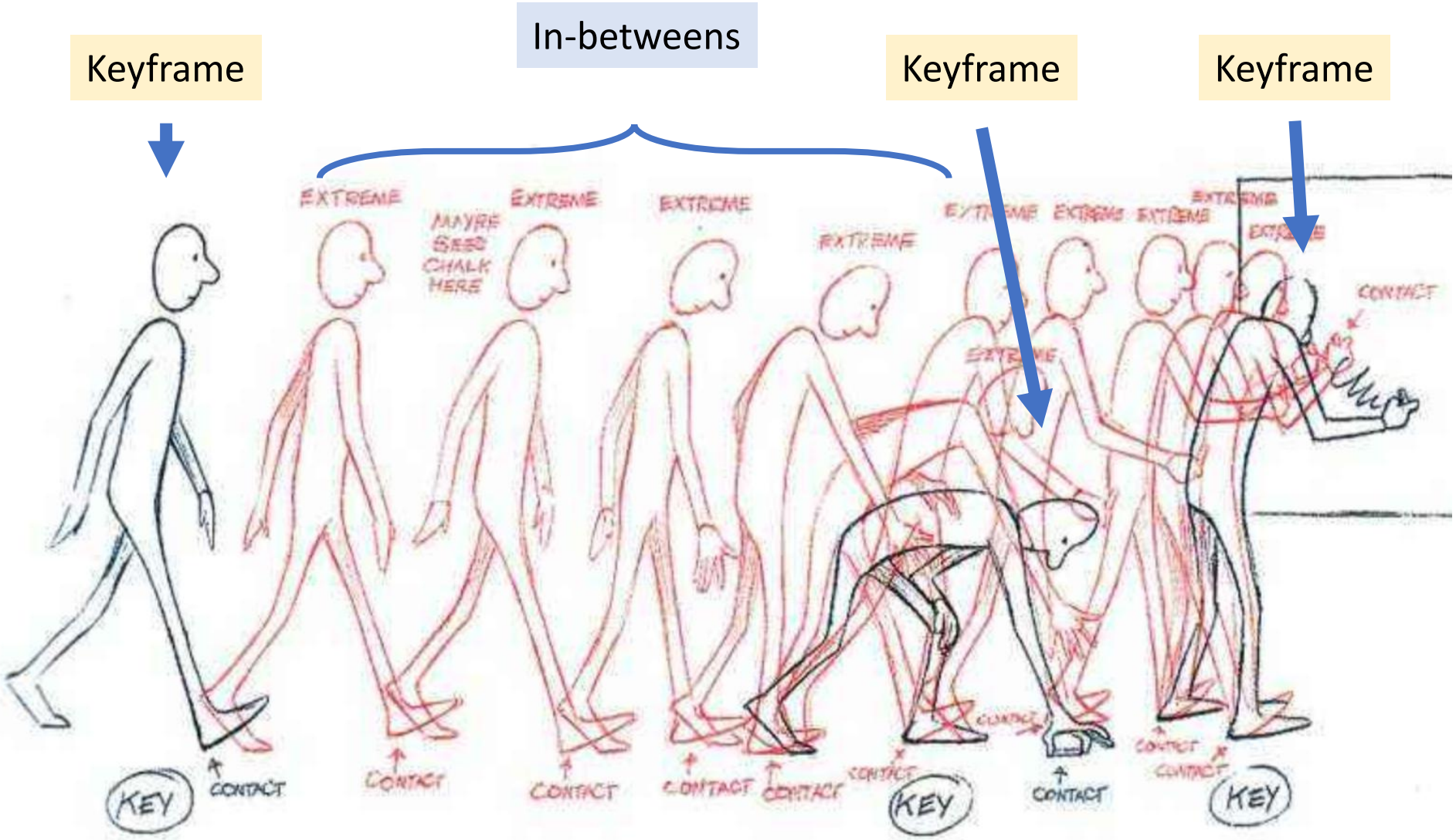
VCL @ PKU

Outline

- Keyframe Animation
  - Interpolation and splines

Origin of Animation: Zoetrope

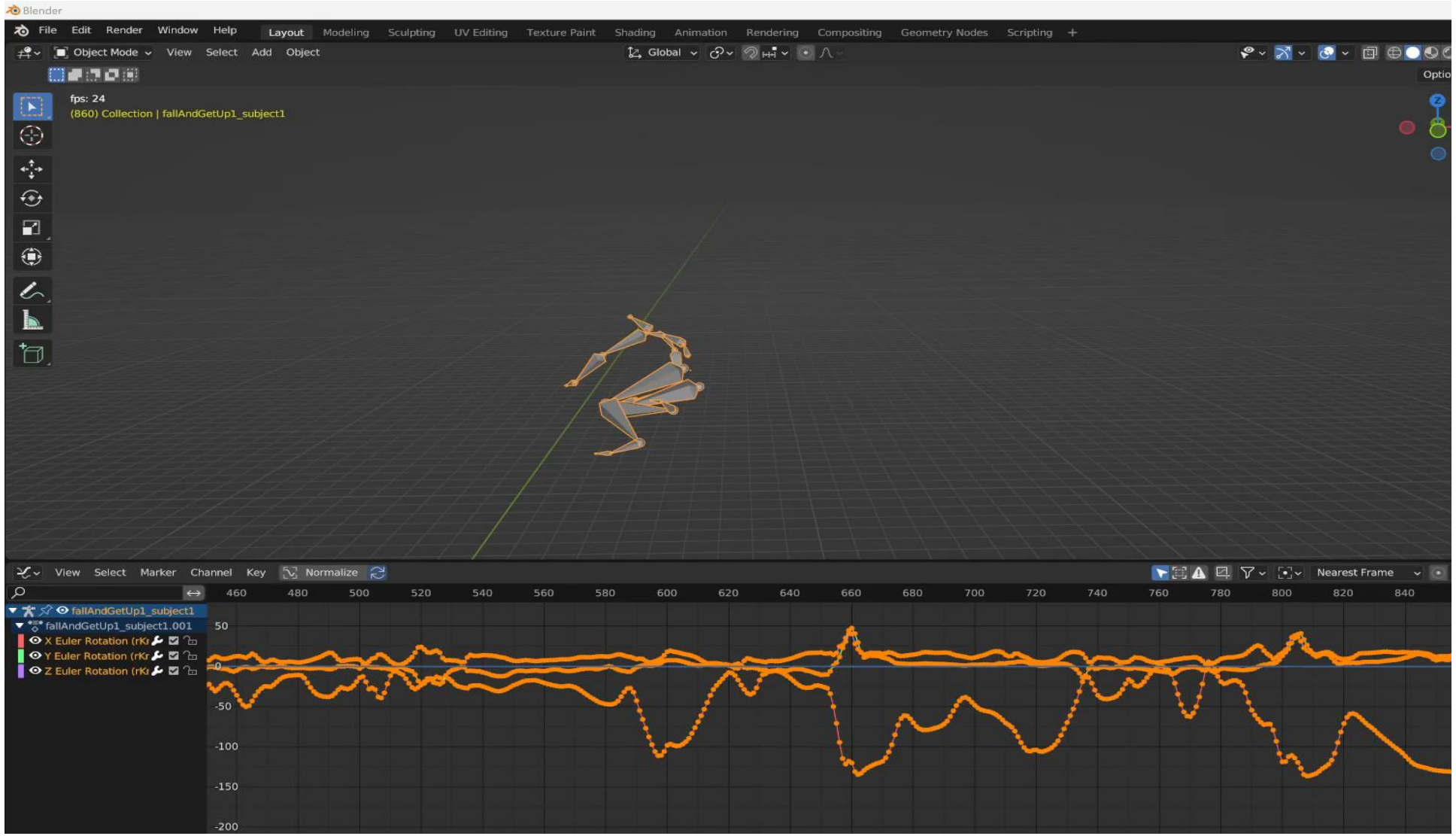
Keyframe Animation



3D Rotoscoping - Keyframe 3D Animation

How to Animate 3D Characters in 1 Minute https://www.youtube.com/watch?v=TjJLIuFKA20

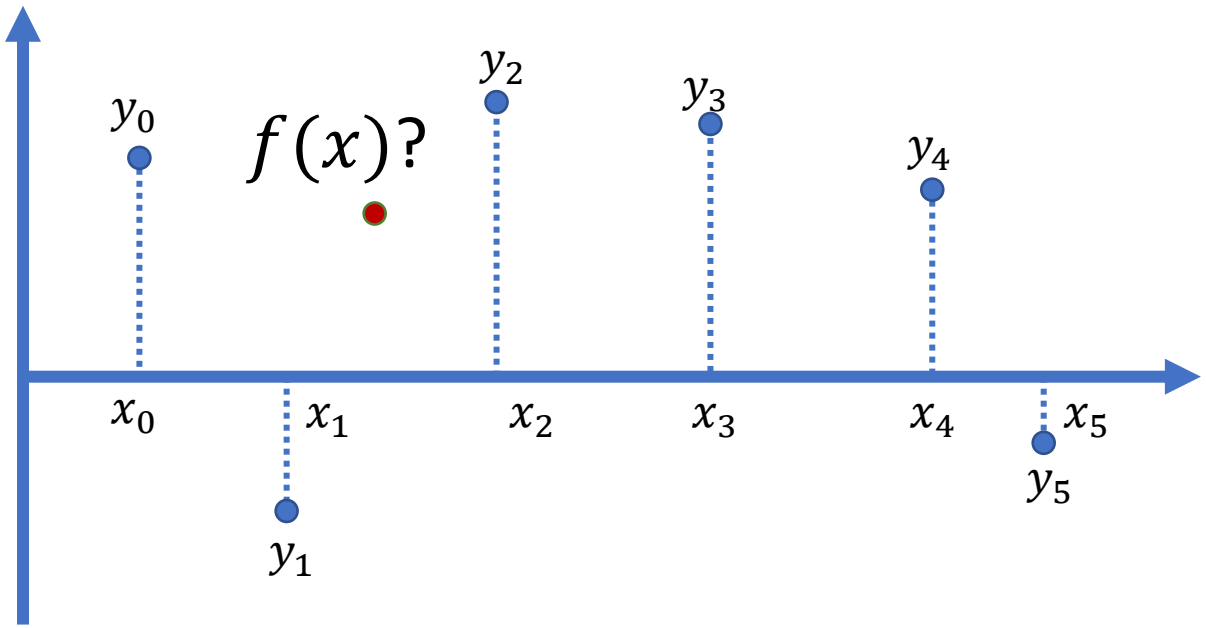
Interpolation Between Keyframes



Interpolation

- Given a set of data pairs $\mathrm{D} = \{(x_i, y_i) | i = 0, \dots, N\}$, find a function $f(x)$ such that

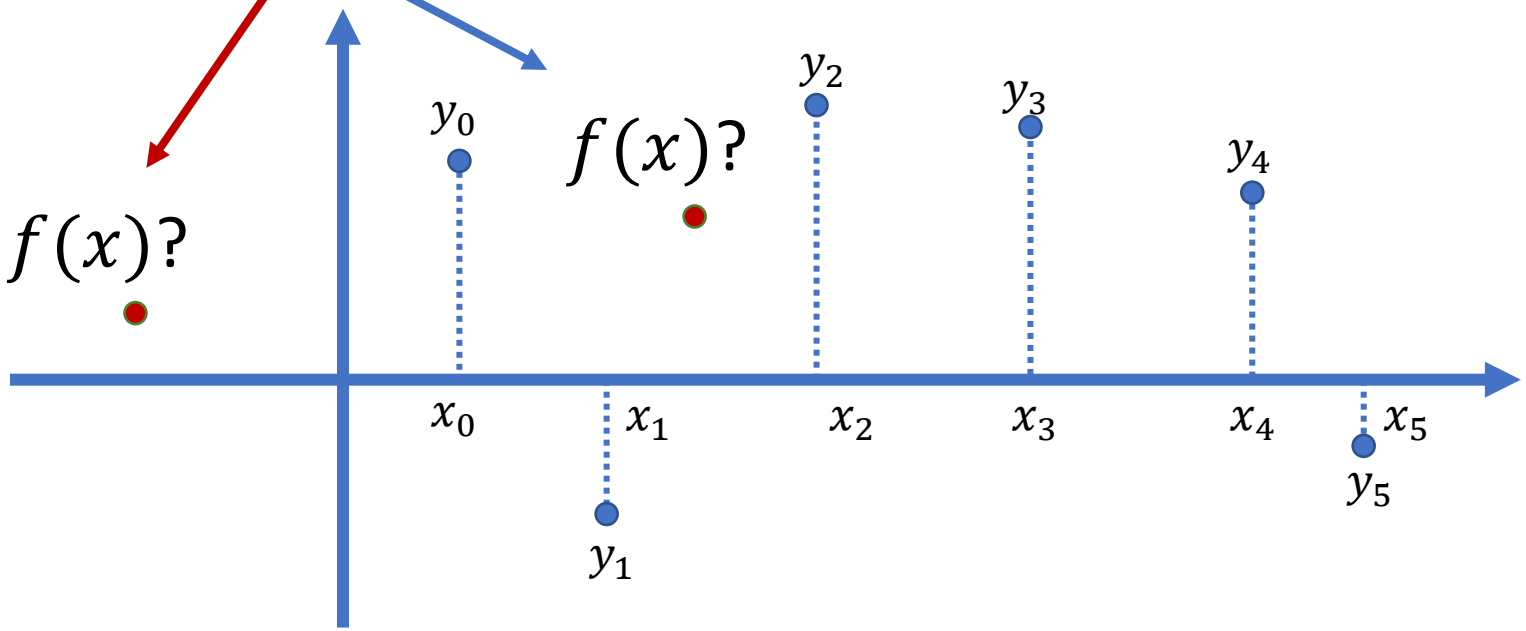
$$f(x_i) = y_i, \forall (x_i, y_i) \in D$$



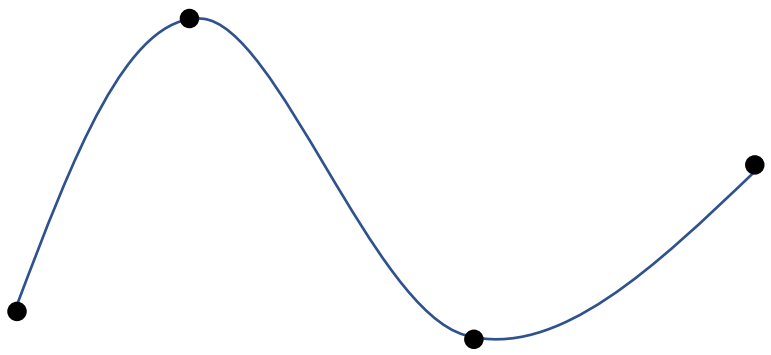
Interpolation

- Interpolation / Extrapolation

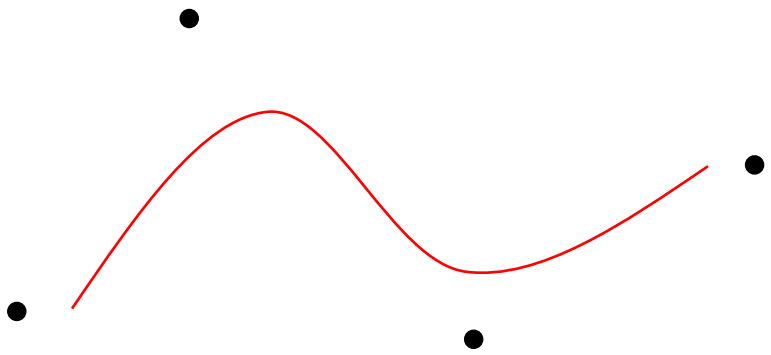
$$f(x_i) = y_i, \forall (x_i, y_i) \in D$$



Interpolation v.s. Fitting



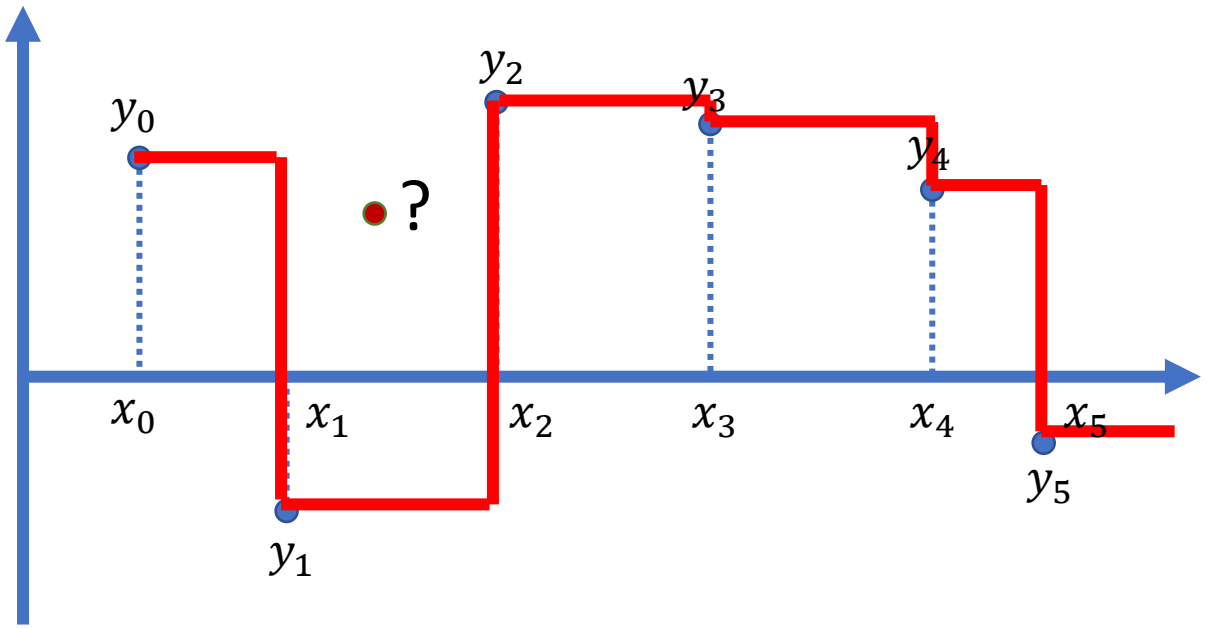
Interpolation



Fitting

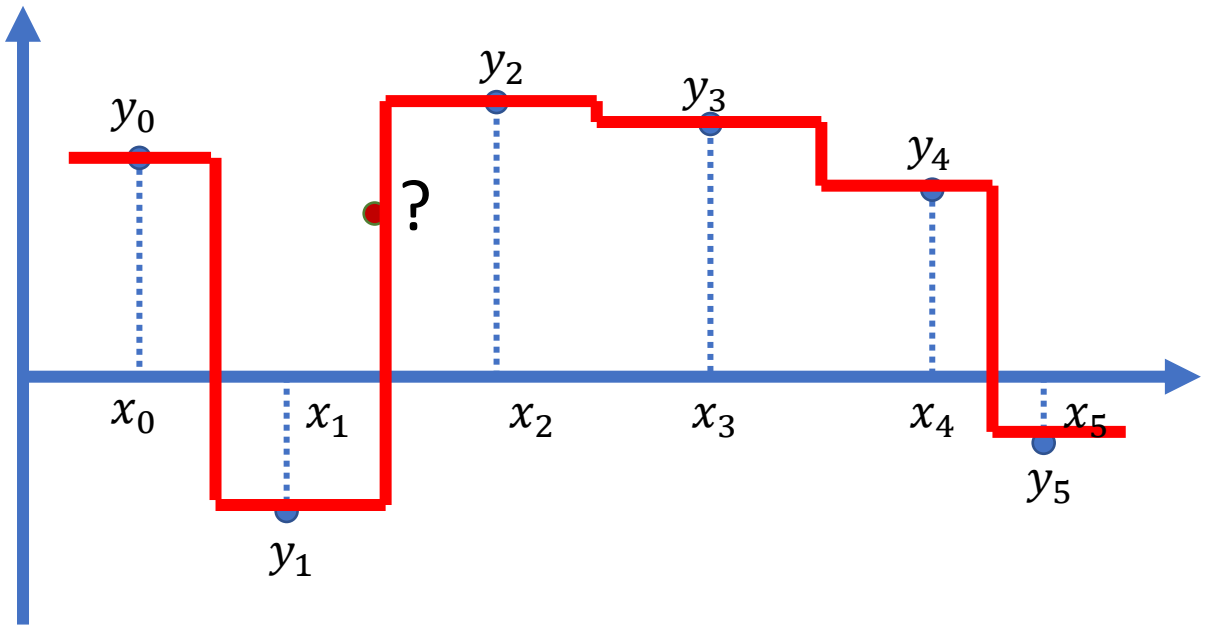
Stepped Interpolation

$$f(x) = y_1$$



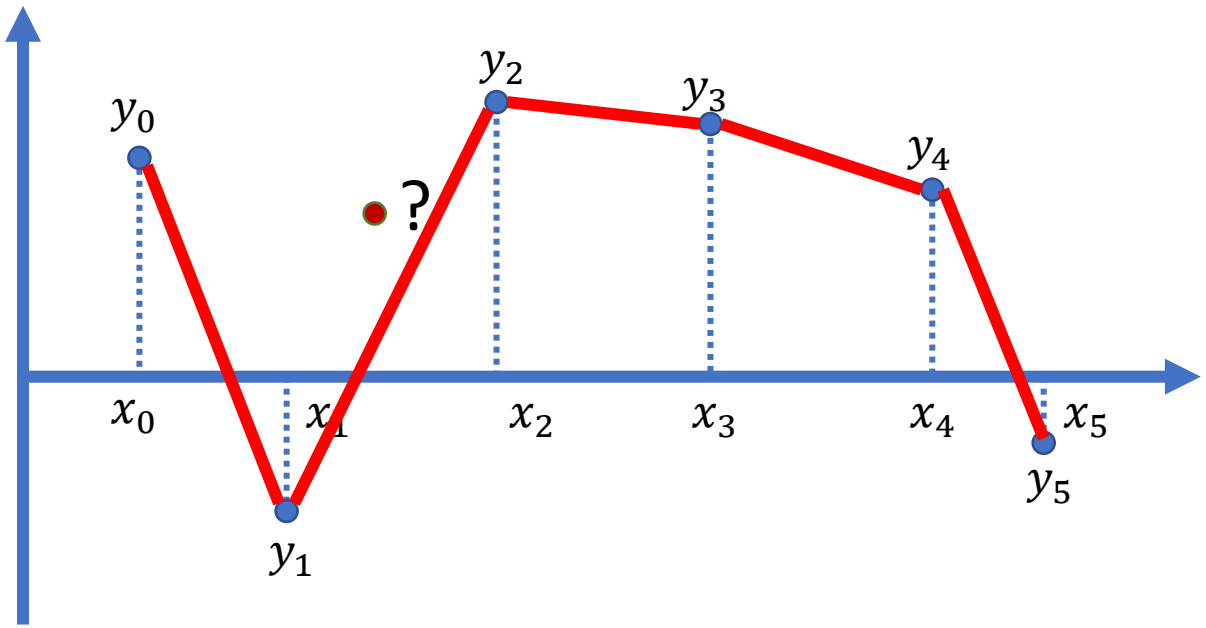
Stepped Interpolation

$$f(x) = y_1$$



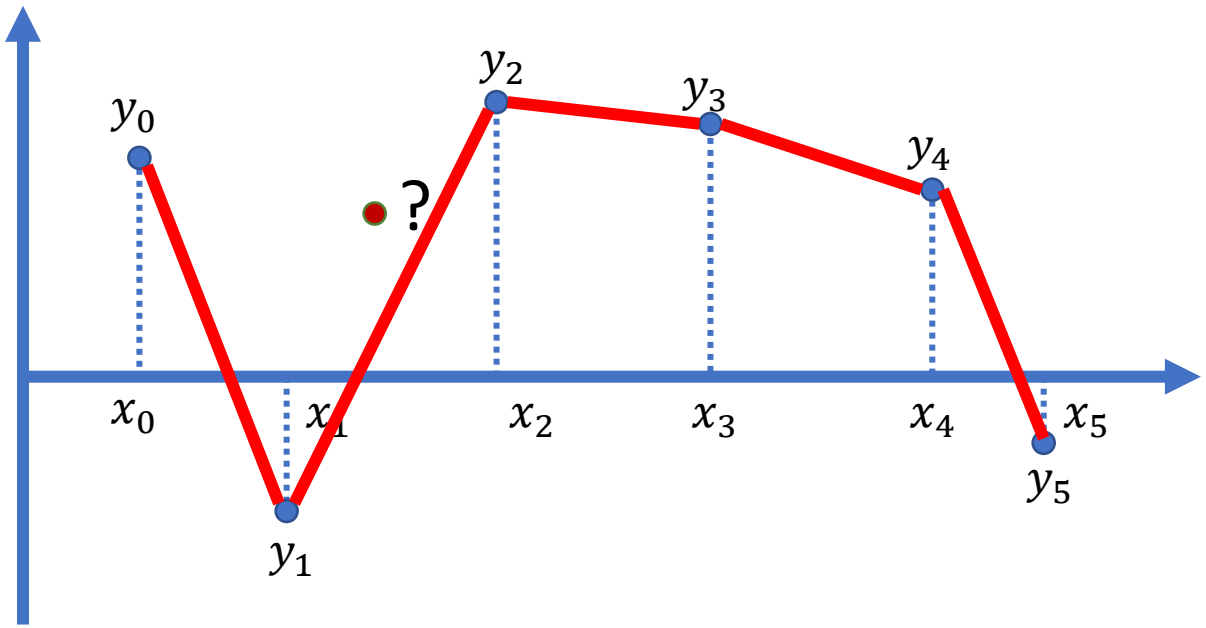
Linear Interpolation

$$f(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1)$$



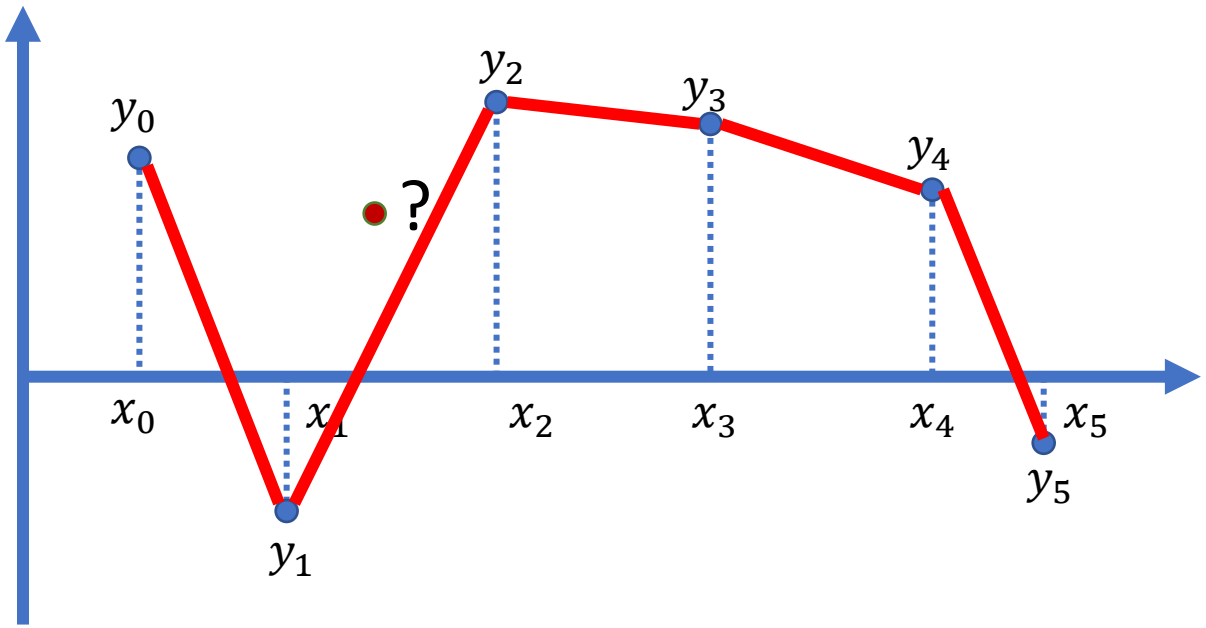
Linear Interpolation

$$f(x) = y_1 + \frac{x - x_1}{x_2 - x_1}(y_2 - y_1)$$



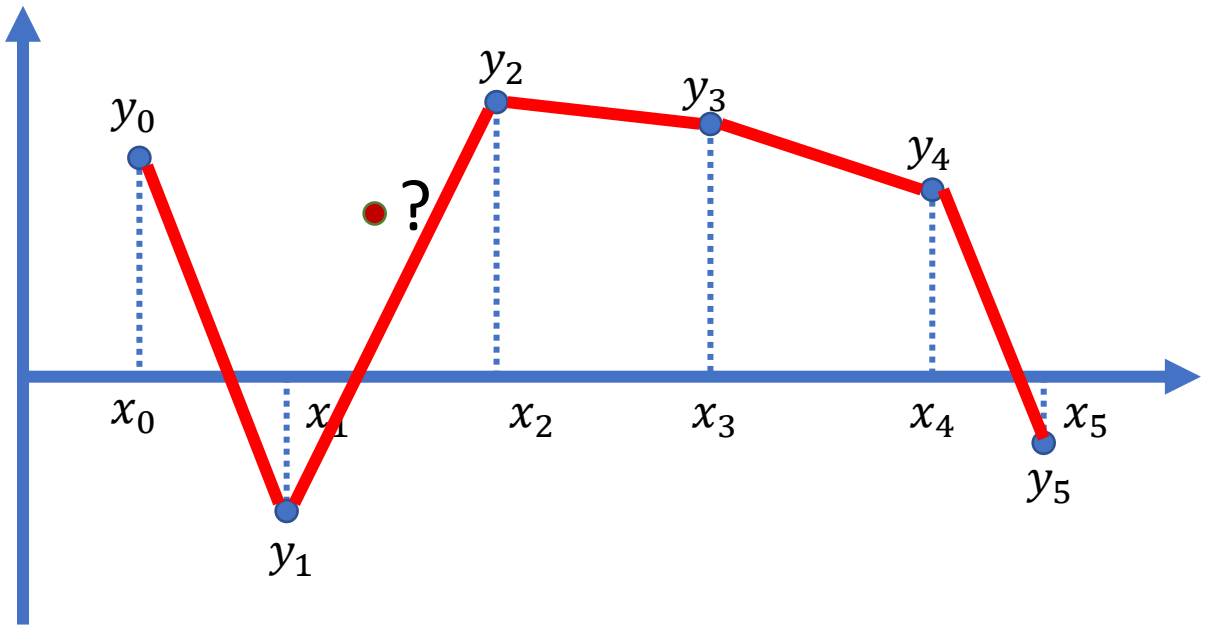
Linear Interpolation

$$f(x) = y_1 + t(y_2 - y_1)$$

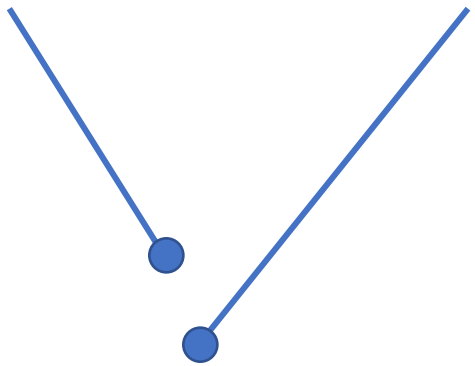


Linear Interpolation

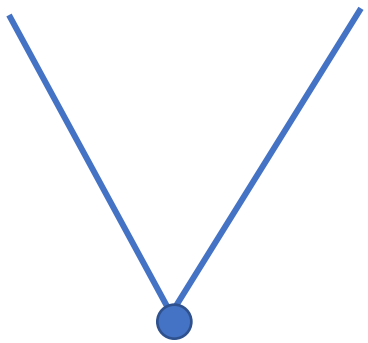
$$f(x) = (1-t)y_1 + ty_2$$



Smoothness



Discontinuity



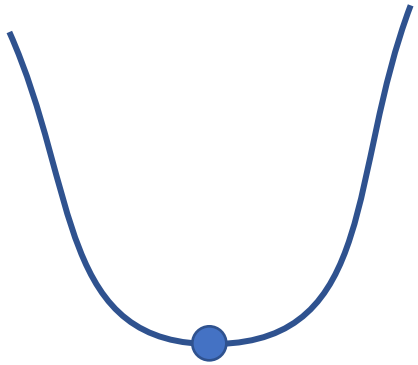
C^0-continuity

positions coincide

C^1-continuity

positions coincide

velocity coincide



C^2-continuity

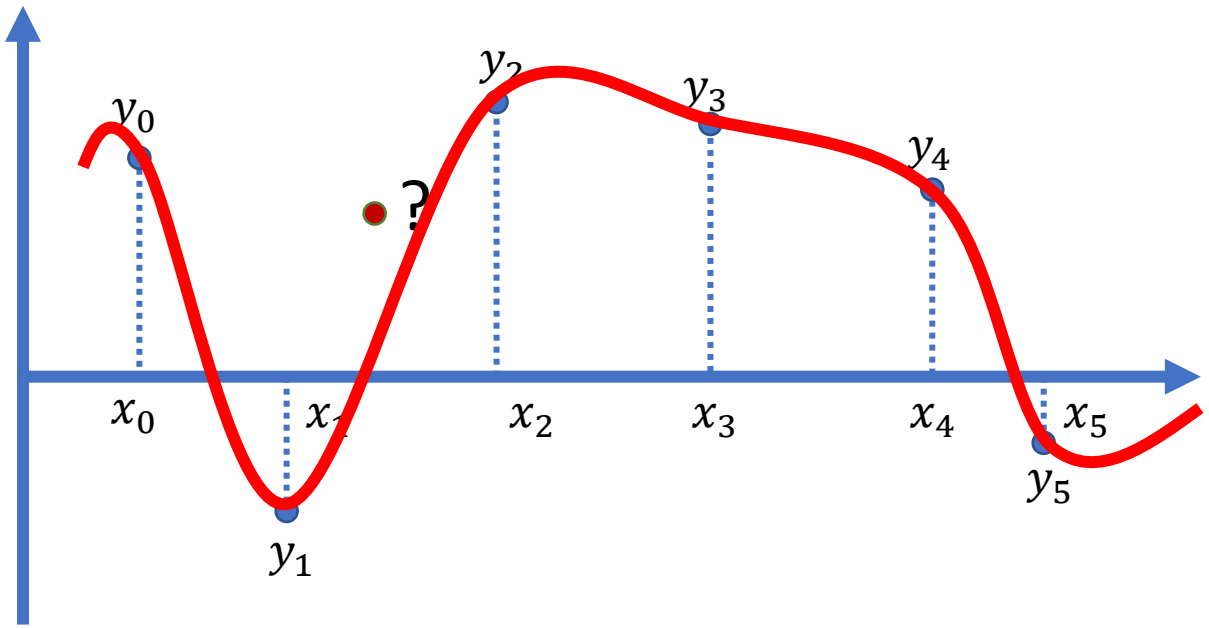
positions coincide

velocity coincide

acceleration coincide

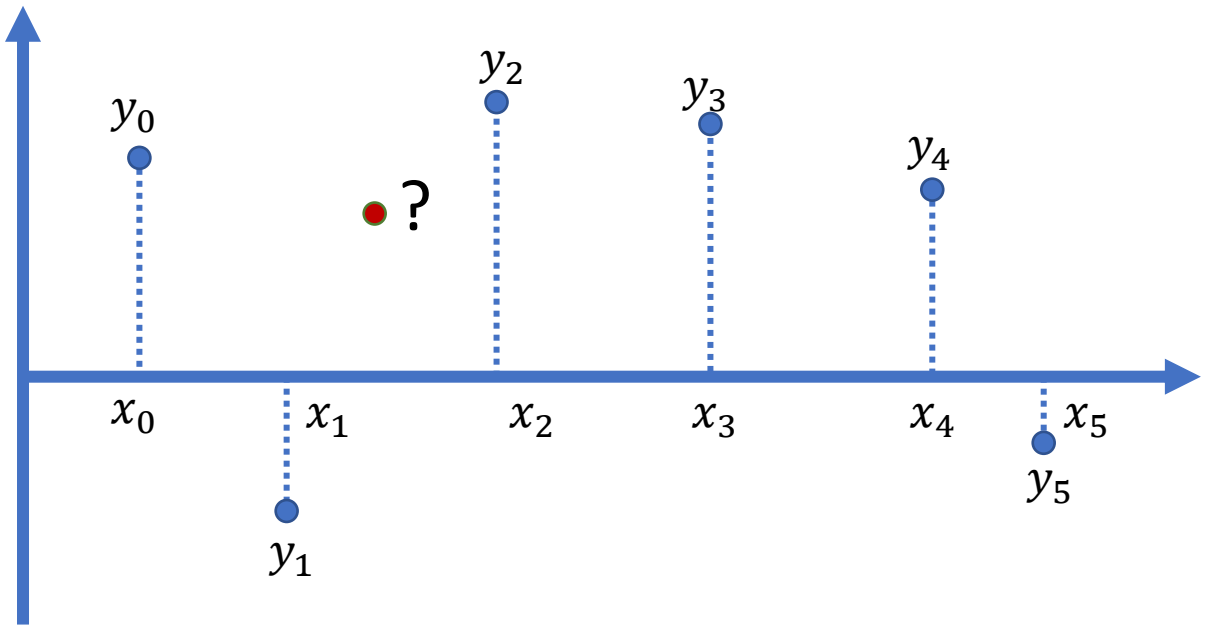
Nonlinear Interpolation?

$$f(x) = ?$$



Polynomial Interpolation

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$



Polynomial Interpolation

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

For any data point in $\mathrm{D} = \{(x_i, y_i) | i = 0, \ldots, N\}$

$$f(x_0) = a_0 + a_1 x_0 + a_2 x_0^2 + \cdots + a_n x_0^n = y_0$$

Polynomial Interpolation

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

For any data point in $\mathrm{D} = \{(x_i, y_i) | i = 0, \ldots, N\}$

$$f(x_0) = a_0 + a_1 x_0 + a_2 x_0^2 + \cdots + a_n x_0^n = y_0$$
$$f(x_1) = a_0 + a_1 x_1 + a_2 x_1^2 + \cdots + a_n x_1^n = y_1$$
$$f(x_2) = a_0 + a_1 x_2 + a_2 x_2^2 + \cdots + a_n x_2^n = y_2$$
$$\ldots\ldots$$
$$f(x_N) = a_0 + a_1 x_N + a_2 x_N^2 + \cdots + a_n x_N^n = y_N$$

Polynomial Interpolation

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

Data point set $\mathrm{D} = \{(x_i, y_i) | i = 0, \ldots, N\}$

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_N \end{bmatrix}$$

Polynomial Interpolation

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

Data point set $\mathrm{D} = \{(x_i, y_i) | i = 0, \ldots, N\}$

$$\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^n \end{bmatrix}^{-1} \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_N \end{bmatrix}$$

Polynomial Interpolation

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

Data point set $\mathrm{D} = \{(x_i, y_i) | i = 0, \ldots, N\}$

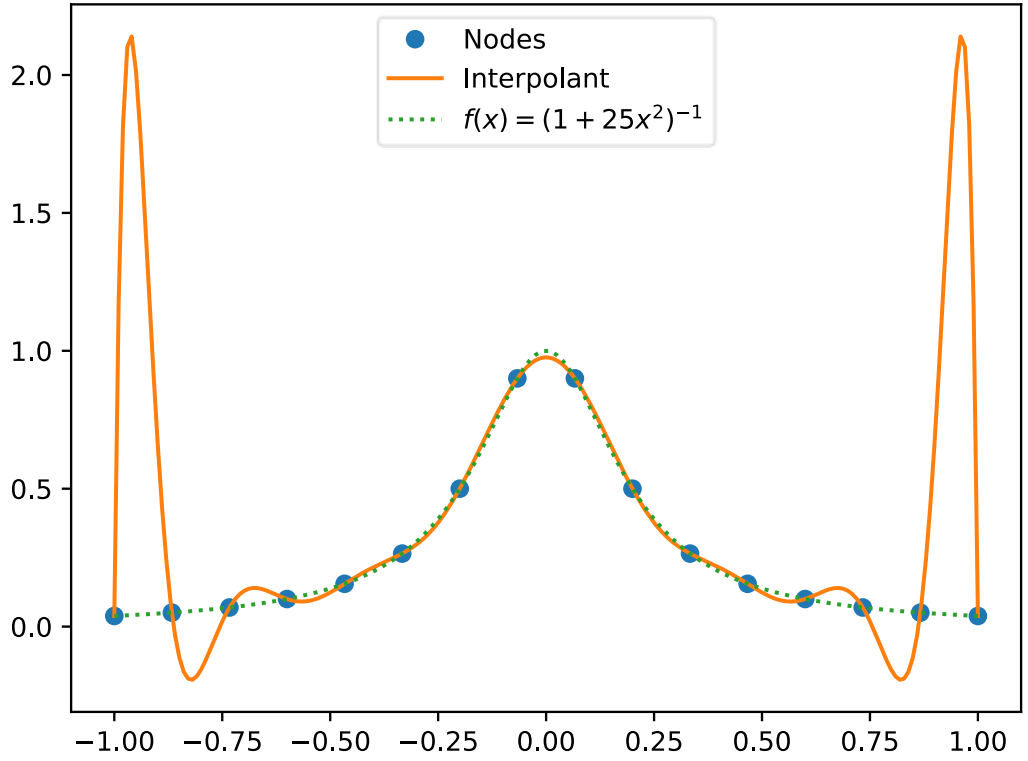
$$\begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^n \end{bmatrix}^{-1} \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_N \end{bmatrix}$$

We need $n = (N - 1)$-degree polynomial to fit N data points!

Polynomial Interpolation

- Runge's phenomenon
  - High-degree polynomial can oscillate at the edges of an interval

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{15} x^{15}$$

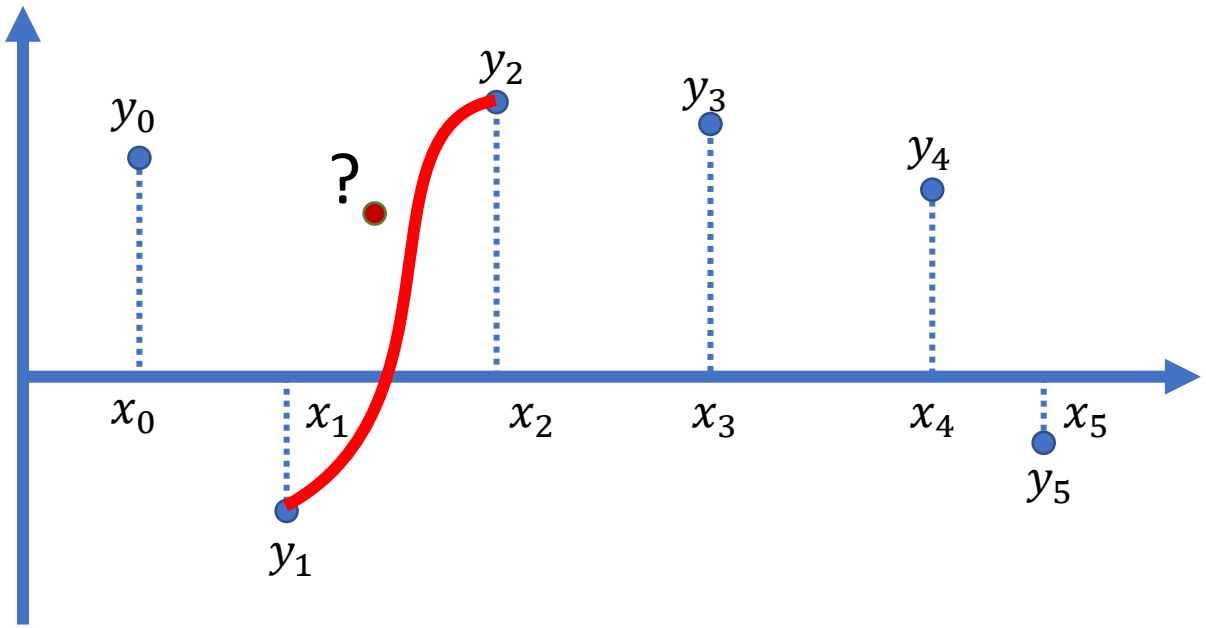


- So low-degree polynomials are preferred
  - But how?

We need $n = (N - 1)$-degree polynomial to fit N data points!

Spline Interpolation

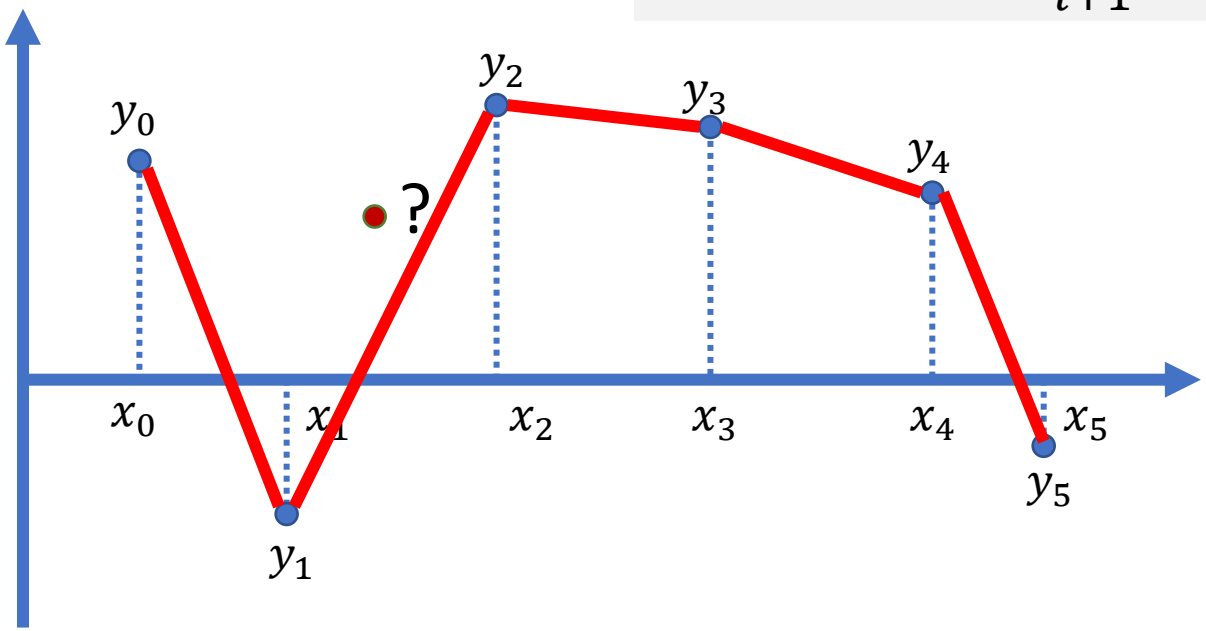
- Interpolation using low-degree piecewise polynomials
  - $f(x) = S_i(x)$, when $x \in [x_i, x_{i+1}]$



Spline Interpolation

- Interpolation using low-degree piecewise polynomials
  - Degree 1 → piecewise linear interpolation

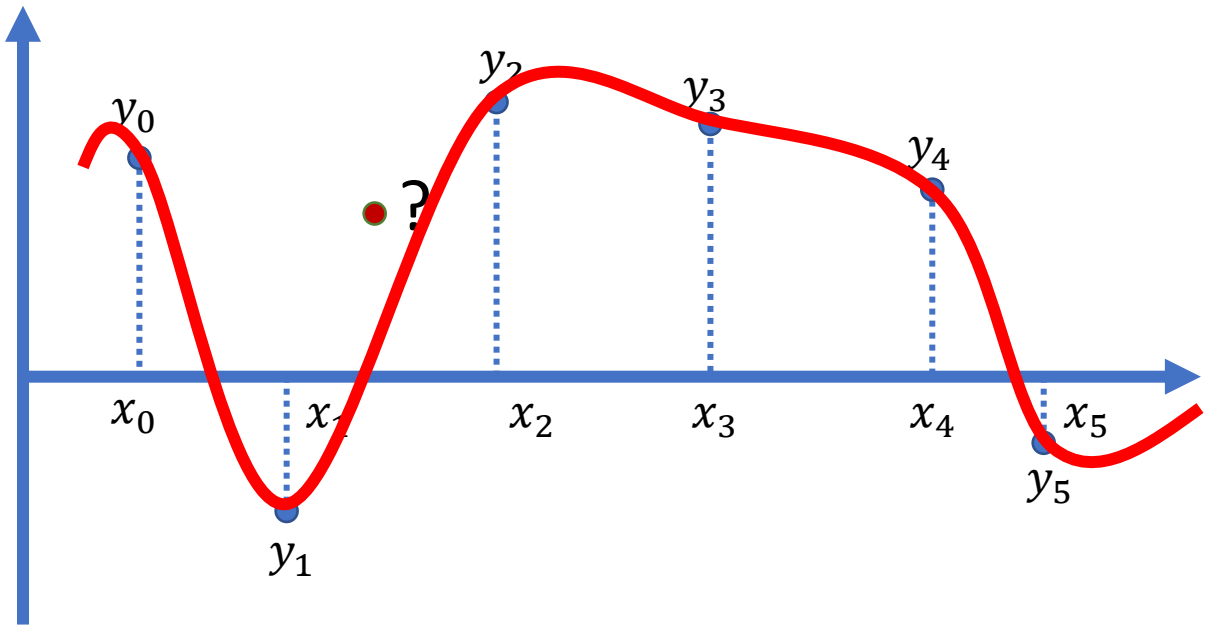
$$S_i(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i)$$



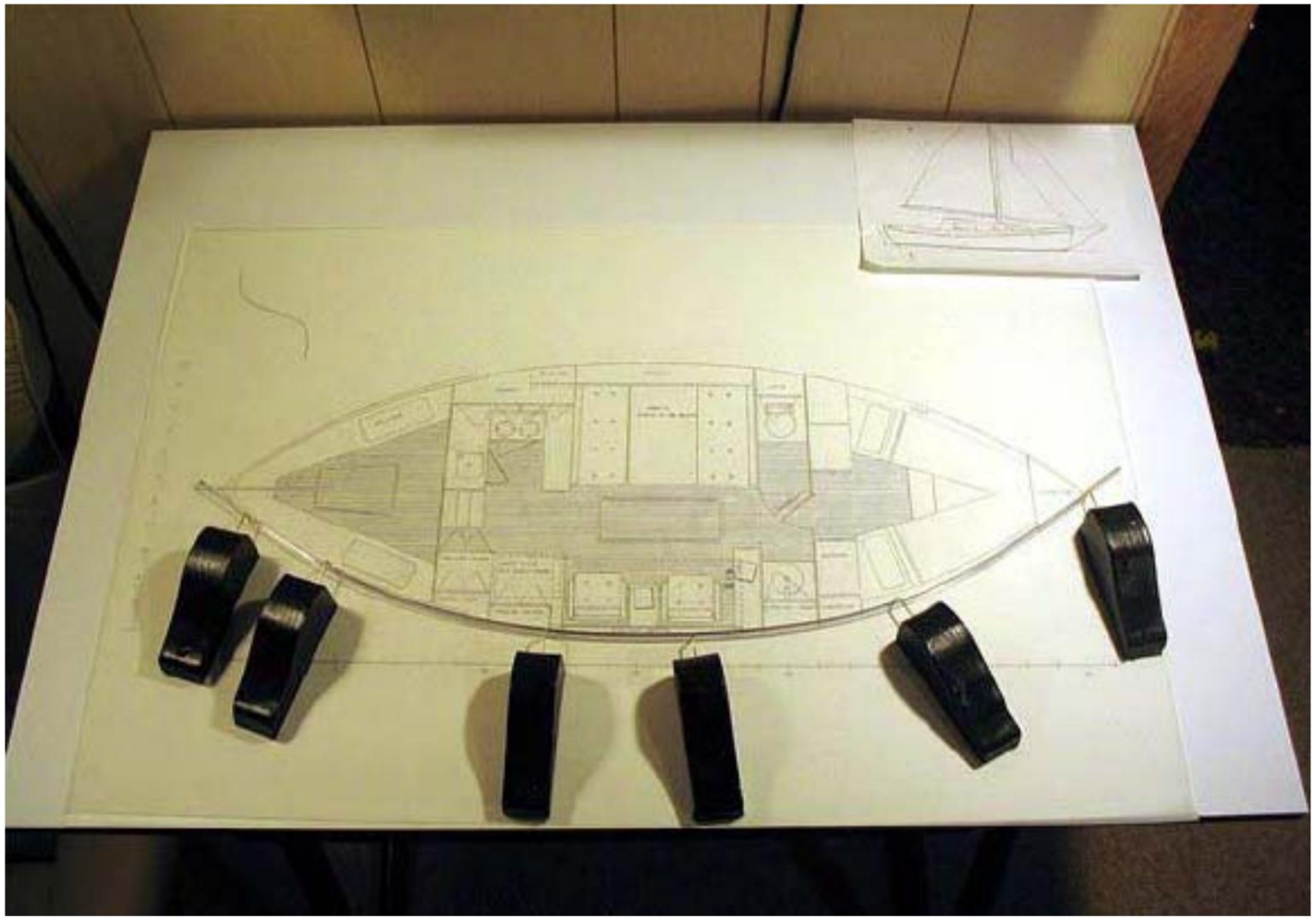
Spline Interpolation

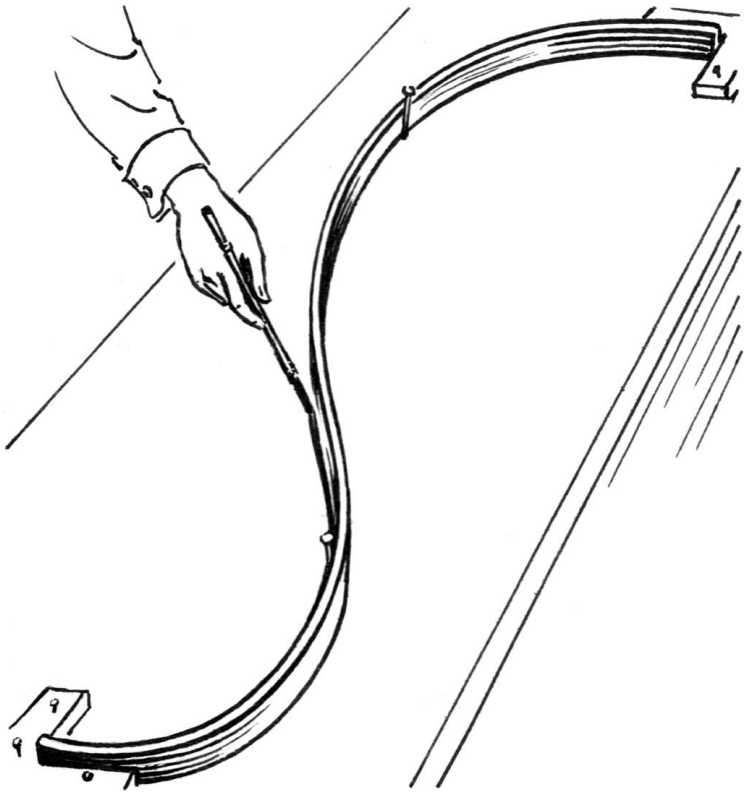
- Interpolation using low-degree piecewise polynomials
  - Third-degree polynomials → Cubic Splines

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$



Spline

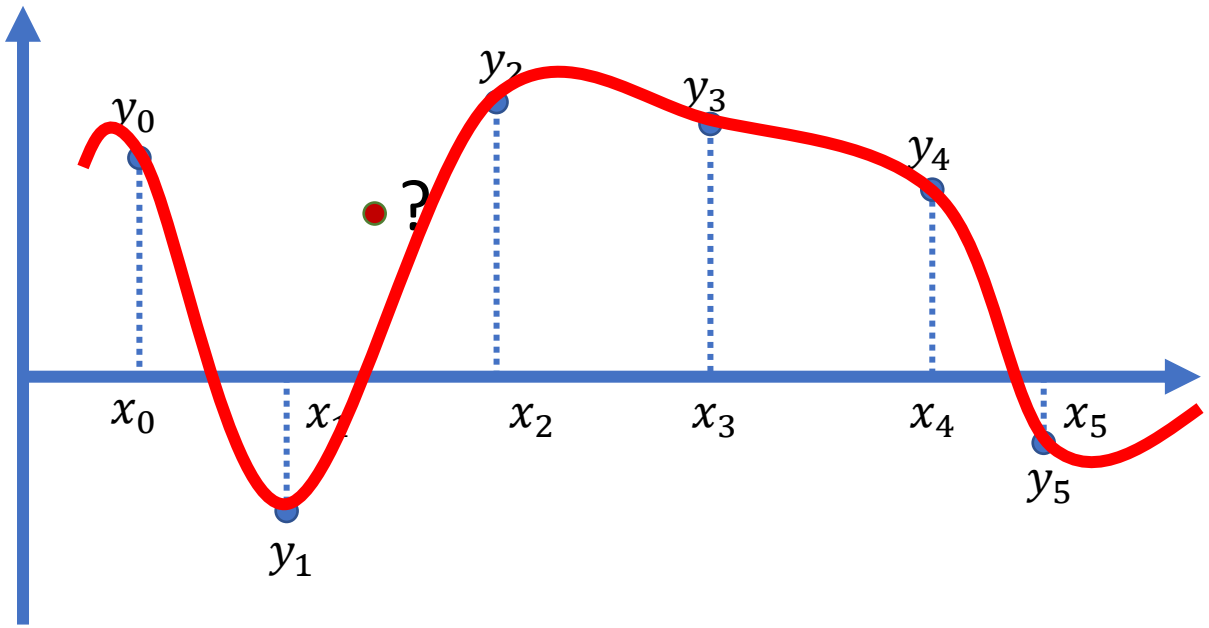




Cubic Splines

- For a set of data points $\mathrm{D} = \{(x_i, y_i) | i = 0, \dots, N\}$

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

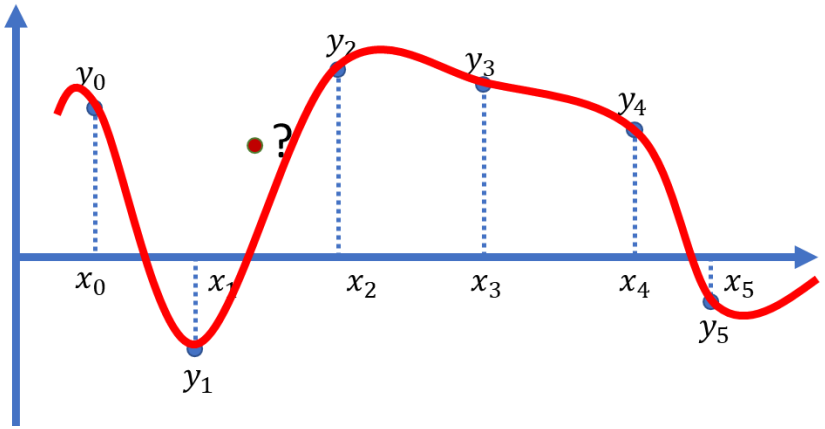


Cubic Splines

- For a set of data points $\mathrm{D} = \{(x_i, y_i) | i = 0, \dots, N\}$

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

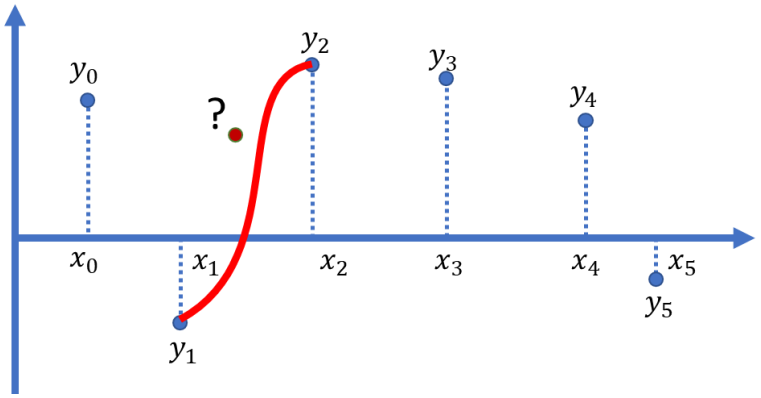
- There are N segments, $4N$ unknown parameters



Cubic Splines

- For a set of data points $\mathrm{D} = \{(x_i, y_i) | i = 0, \dots, N\}$

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$



- There are N segments, $4N$ unknown parameters

Interpolation condition: $S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}$

C^1 continuity: $S'_{i-1}(x_i) = S'_i(x_i)$

C^2 continuity: $S''_{i-1}(x_i) = S''_i(x_i)$

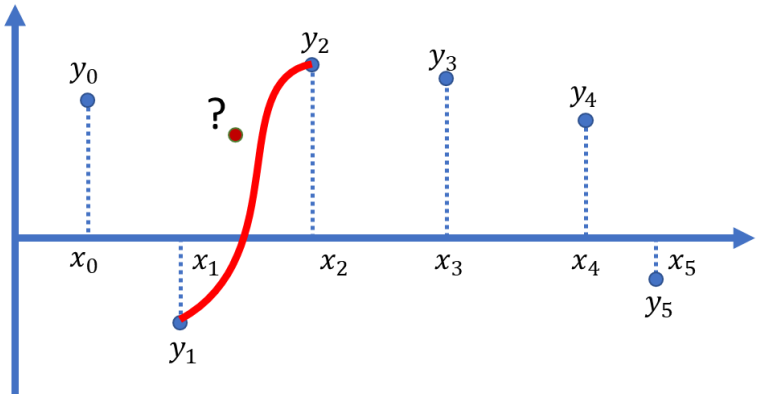
boundary condition: $S'_0(x_0), S'_{n-1}(x_n), S''_0(x_0), S''_{n-1}(x_n)$

Linear Equation

Cubic Splines

- For a set of data points $\mathrm{D} = \{(x_i, y_i) | i = 0, \dots, N\}$

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$



- There are N segments, $4N$ unknown parameters

Interpolation condition: $S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}$

No local control: Every data point affects the entire curve

C^1 continuity: $S_{i-1}'(x_i) = S_i'(x_i)$

Computationally expensive: Need to solve very large linear system when N is big

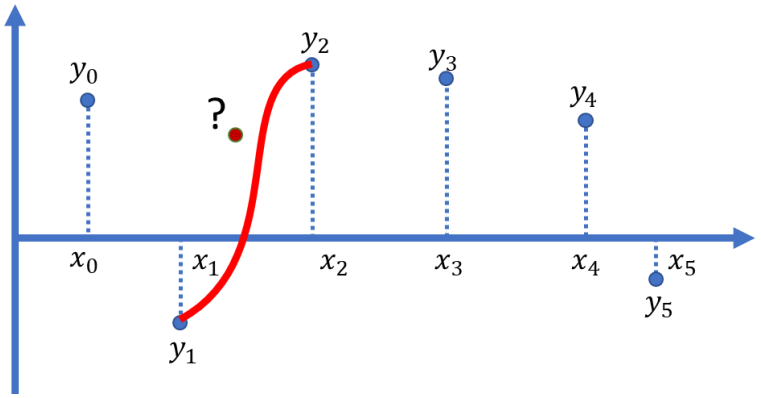
Boundary condition: $S_0(x_0), S_{n-1}(x_n), S_0'(x_0), S_{n-1}'(x_n)$

Linear Equation

Cubic Hermite Splines

- For a set of data points $\mathrm{D} = \{(x_i, y_i) | i = 0, \dots, N\}$

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$



Interpolation condition: $S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}$

C^1 continuity: $S'_{i-1}(x_i) = S'_i(x_i)$

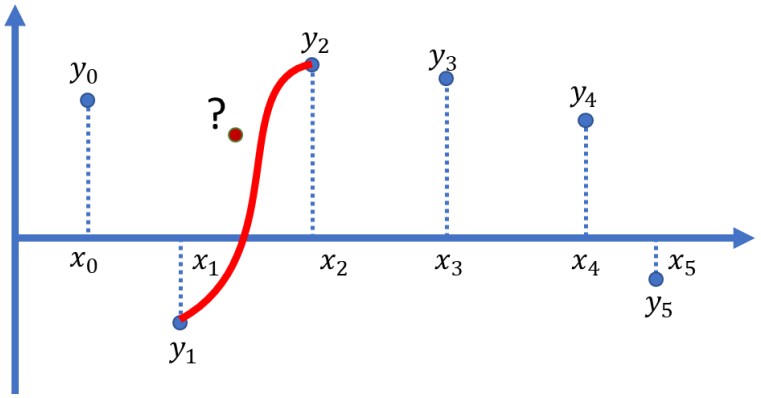
C^2 continuity: $S''_{i-1}(x_i) = S''_i(x_i)$

boundary condition: $S'_0(x_0), S'_{n-1}(x_n), S''_0(x_0), S''_{n-1}(x_n)$

Cubic Hermite Splines

- For a set of data points $\mathrm{D} = \{(x_i, y_i) | i = 0, \dots, N\}$

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$



Interpolation condition: $S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}$

~~C^1 continuity: $S'_{i-1}(x_i) = S'_i(x_i)$~~

~~C^2 continuity: $S''_{i-1}(x_i) = S''_i(x_i)$~~

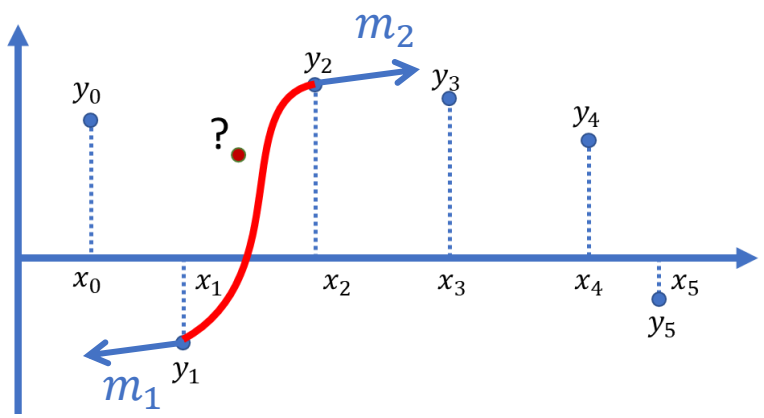
~~boundary condition: $S'_0(x_0), S'_{n-1}(x_n), S''_0(x_0), S''_{n-1}(x_n)$~~

Cubic Hermite Splines

- For a set of data points $\mathrm{D} = \{(x_i, y_i) | i = 0, \dots, N\}$

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

- also we know the first derivatives
  $D' = \{(x_i, m_i) | i = 0, \dots, N\}, S_i' = m_i$



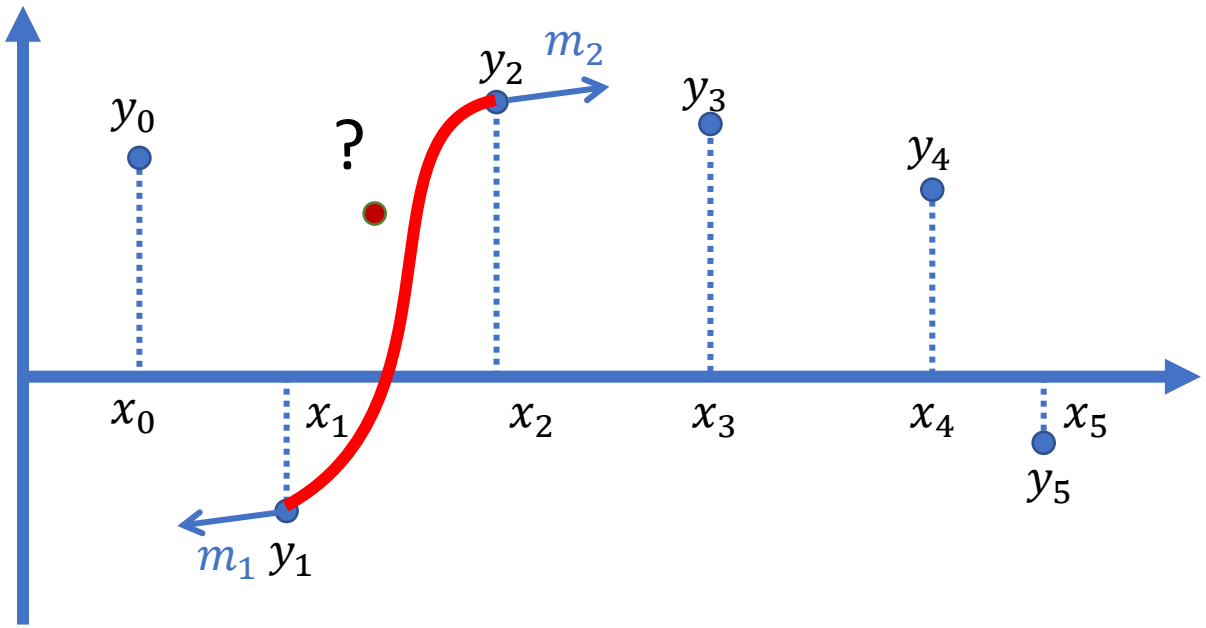
For each segment i, $\quad S_i(x_i) = y_i, \quad S_i(x_{i+1}) = y_{i+1}$

Interpolation condition: $\quad S_i'(x_i) = m_i, \quad S_i'(x_{i+1}) = m_{i+1}$

Cubic Hermite Interpolation

Given $x_1, y_1, m_1, x_2, y_2, m_2$, we need to compute a cubic curve

$$S(x) = ax^3 + bx^2 + cx + d$$



Cubic Hermite Interpolation

Given $x_1, y_1, m_1, x_2, y_2, m_2$, we need to compute a cubic curve

$$S(x) = ax^3 + bx^2 + cx + d$$

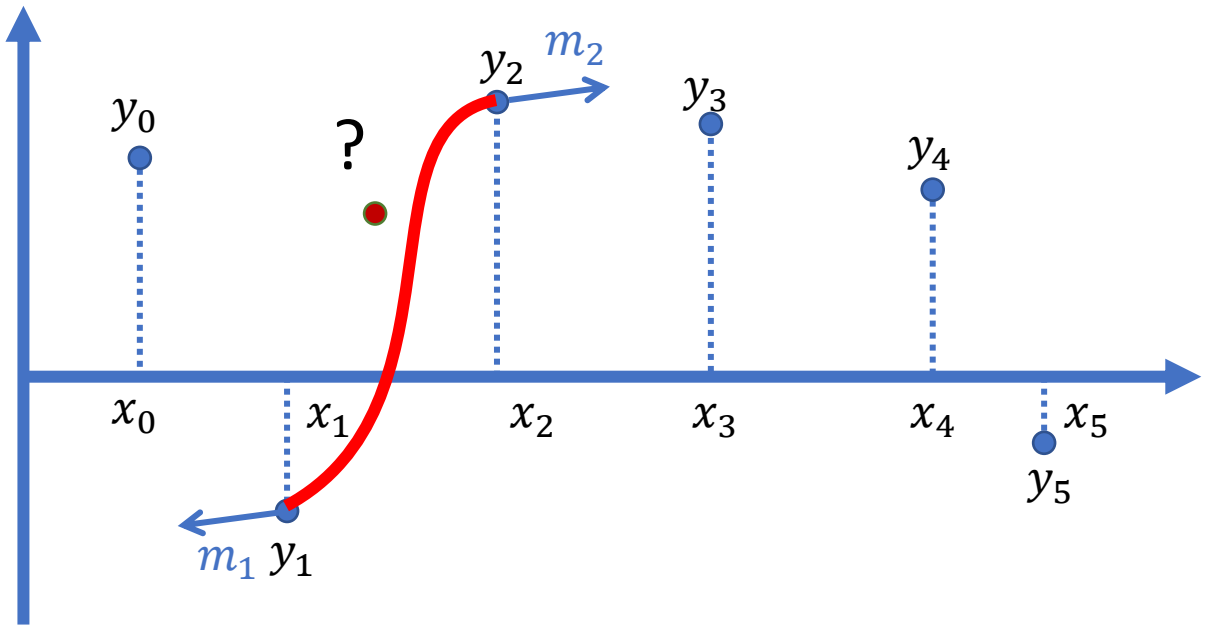
such that

$$S(x_1) = y_1$$

$$S(x_2) = y_2$$

$$S'(x_1) = m_1$$

$$S'(x_2) = m_2$$



Cubic Hermite Interpolation

Given $x_1, y_1, m_1, x_2, y_2, m_2$, we need to compute a cubic curve

$$S(t) = at^3 + bt^2 + ct + d \qquad t = \frac{x - x_1}{x_2 - x_1}$$

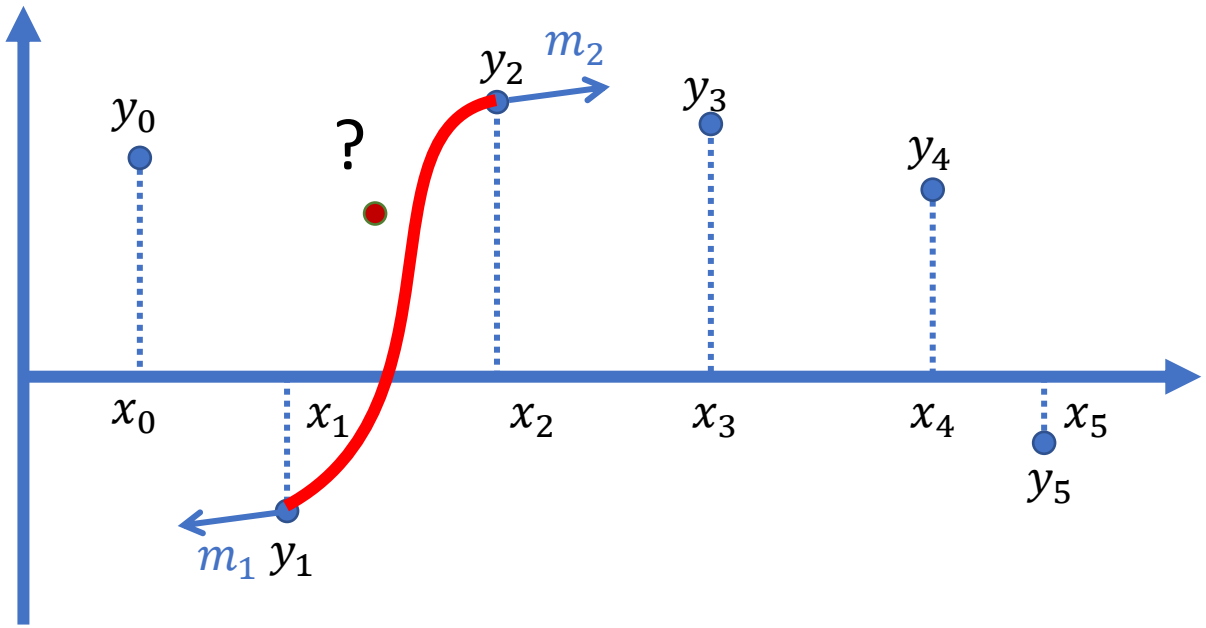
such that

$$S(0) = y_1$$

$$S(1) = y_2$$

$$S'(0) = m_1$$

$$S'(1) = m_2$$



Cubic Hermite Interpolation

Given $x_1, y_1, m_1, x_2, y_2, m_2$, we need to compute a cubic curve

$$S(t) = at^3 + bt^2 + ct + d \qquad t = \frac{x - x_1}{x_2 - x_1}$$

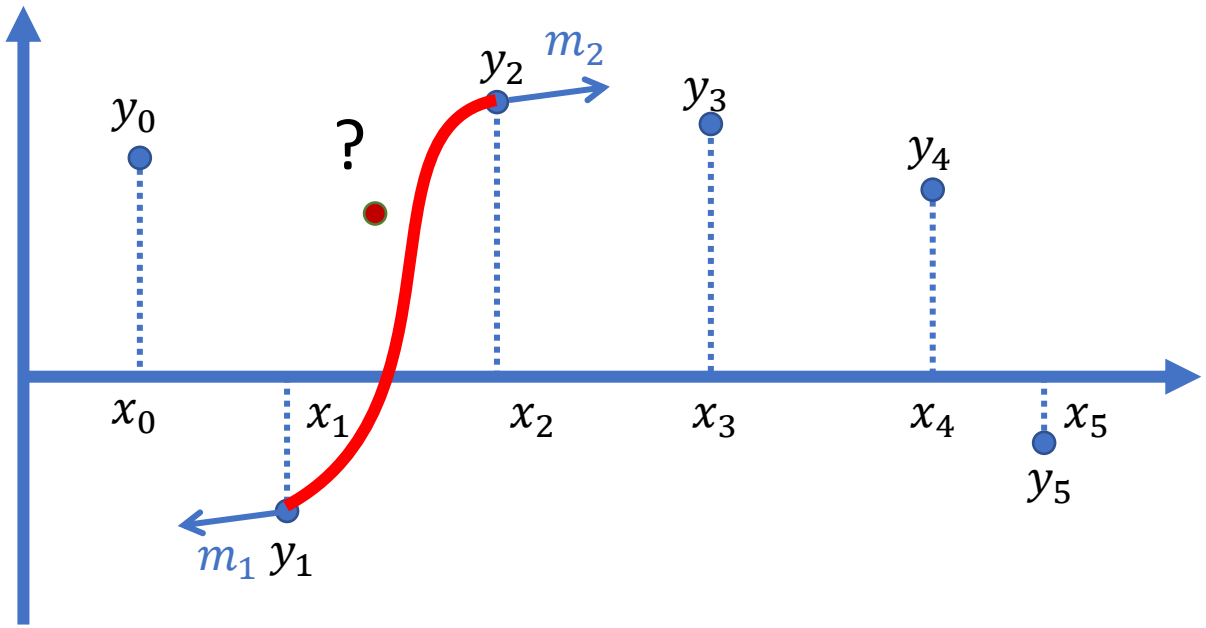
such that

$$S(0) = y_1$$

$$S(1) = y_2$$

$$S'(0) = m_1$$

$$S'(1) = m_2$$



Cubic Hermite Interpolation

Given $x_1, y_1, m_1, x_2, y_2, m_2$, we need to compute a cubic curve

$$S(t) = at^3 + bt^2 + ct + d \qquad t = \frac{x - x_1}{x_2 - x_1}$$

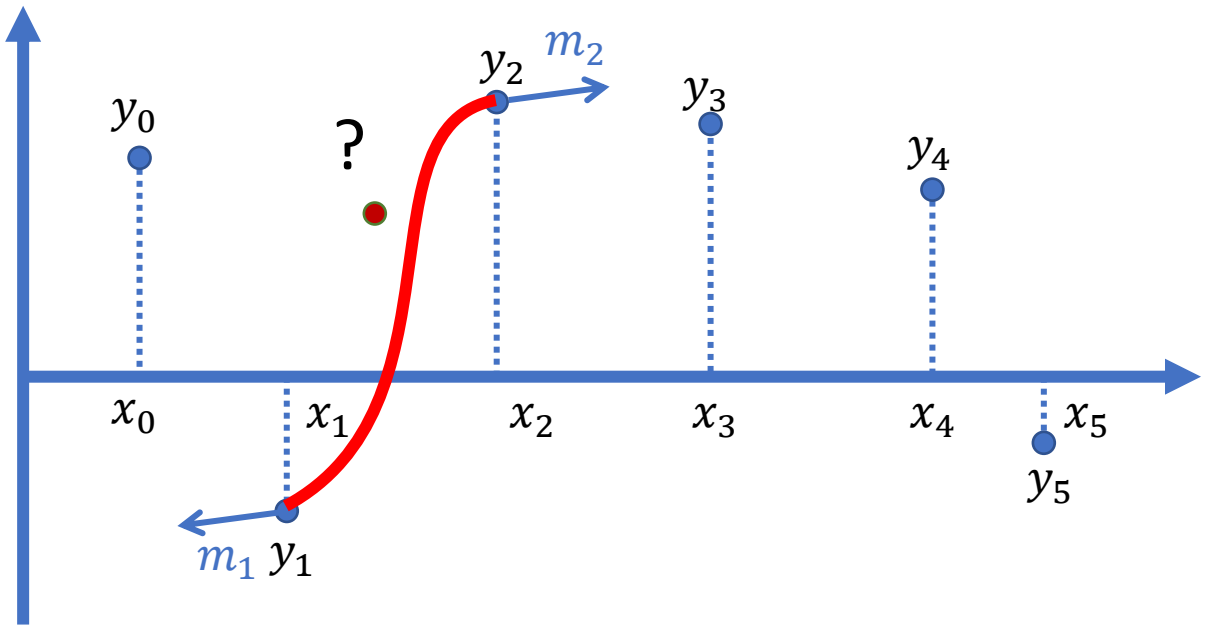
such that

$$S(0) = y_1 = d$$

$$S(1) = y_2 = a + b + c + d$$

$$S'(0) = m_1 = c$$

$$S'(1) = m_2 = 3a + 2b + c$$



Cubic Hermite Interpolation

Given $x_1, y_1, m_1, x_2, y_2, m_2$, we need to compute a cubic curve

$$S(t) = at^3 + bt^2 + ct + d \qquad t = \frac{x - x_1}{x_2 - x_1}$$

such that

$$S(0) = y_1 = d$$

$$S(1) = y_2 = a + b + c + d$$

$$S'(0) = m_1 = c$$

$$S'(1) = m_2 = 3a + 2b + c$$

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

Cubic Hermite Interpolation

Given $x_1, y_1, m_1, x_2, y_2, m_2$, we need to compute a cubic curve

$$S(t) = at^3 + bt^2 + ct + d \qquad t = \frac{x - x_1}{x_2 - x_1}$$

such that

$$S(0) = y_1 = d$$

$$S(1) = y_2 = a + b + c + d$$

$$S'(0) = m_1 = c$$

$$S'(1) = m_2 = 3a + 2b + c$$

$$\Rightarrow \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

Cubic Hermite Interpolation

Given $x_1, y_1, m_1, x_2, y_2, m_2$, we have a cubic curve

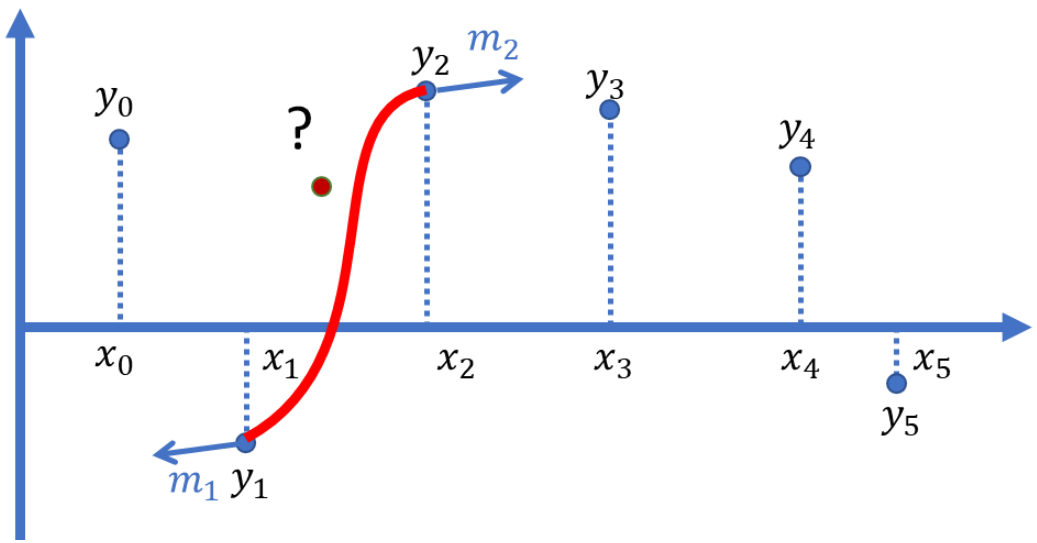
$$S(t) = at^3 + bt^2 + ct + d \qquad t = \frac{x - x_1}{x_2 - x_1}$$

where

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

and

$$S(0) = y_i, \quad S(1) = y_{i+1}$$
$$S(0) = m_i, \quad S(1) = m_{i+1}$$



Cubic Hermite Interpolation

$$S(t) = at^3 + bt^2 + ct + d$$

$$= \begin{bmatrix} t^3 & t^2 & t^1 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

$$= \begin{bmatrix} t^3 & t^2 & t^1 & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

Cubic Hermite Interpolation

$$S(t) = at^3 + bt^2 + ct + d$$

$$= \begin{bmatrix} t^3 & t^2 & t^1 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}$$

$$= \begin{bmatrix} t^3 & t^2 & t^1 & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

Cubic Hermite Interpolation

$$S(t) = at^3 + bt^2 + ct + d$$

$$= \begin{bmatrix} t^3 & t^2 & t^1 & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

$$= \begin{bmatrix} 2t^3 - 3t^2 + 1 \\ -2t^3 + 3t^2 \\ t^3 - 2t^2 + t \\ t^3 - t^2 \end{bmatrix}^T \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

Hermite Basis Functions

$$S(t) = at^3 + bt^2 + ct + d$$

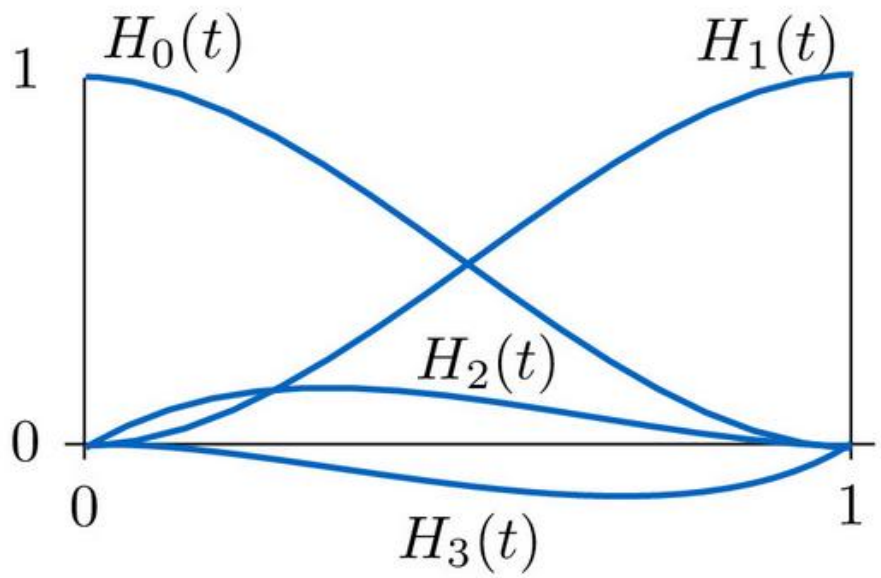
$$= \begin{bmatrix} H_0(t) & H_1(t) & H_2(t) & H_3(t) \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

$$H_0(t) = 2t^3 - 3t^2 + 1$$

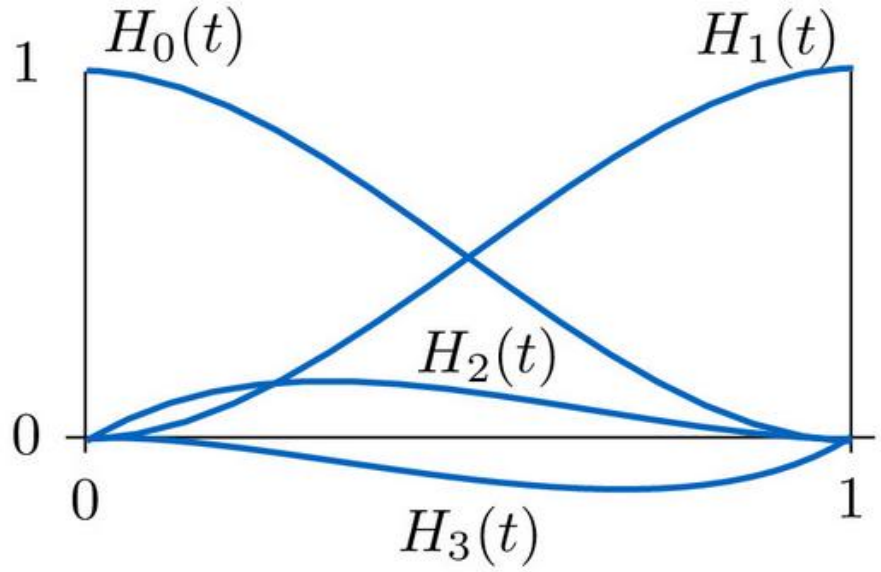
$$H_1(t) = -2t^3 + 3t^2$$

$$H_2(t) = t^3 - 2t^2 + t$$

$$H_3(t) = t^3 - t^2$$



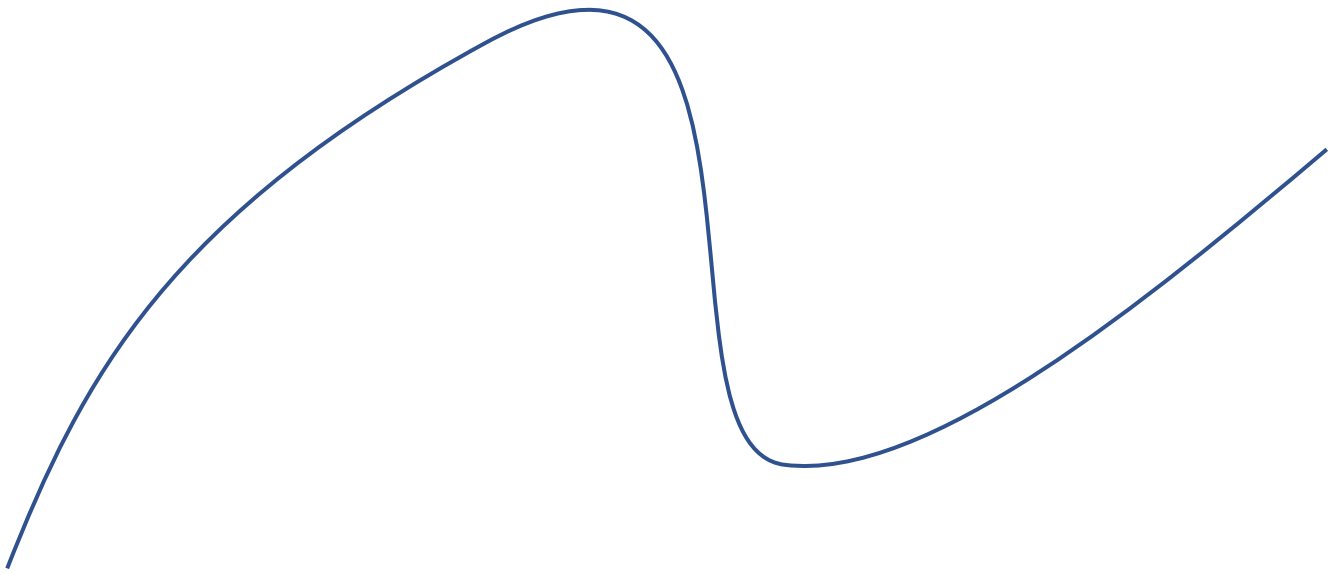
Generalization to Higher Dimensionality



$$\boldsymbol{S}(t) = \boldsymbol{a}t^3 + \boldsymbol{b}t^2 + \boldsymbol{c}t + \boldsymbol{d}$$

$$= \begin{bmatrix} \vdots & \vdots & \vdots & \vdots \\ \boldsymbol{y}_1 & \boldsymbol{y}_2 & \boldsymbol{m}_1 & \boldsymbol{m}_2 \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} H_0(t) \\ H_1(t) \\ H_2(t) \\ H_3(t) \end{bmatrix}$$

Example: Curve Tool of PowerPoint



Catmull-Rom Spline

$$S(t) = at^3 + bt^2 + ct + d$$

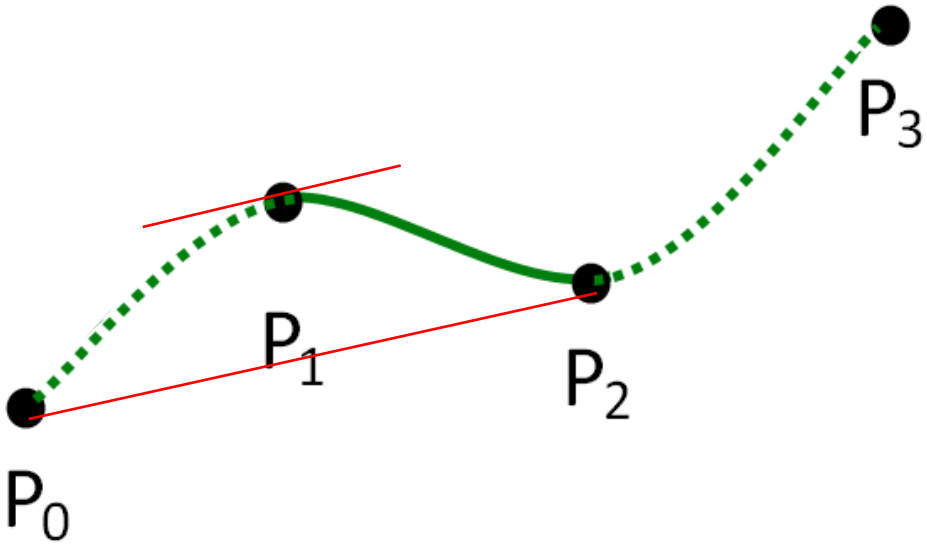
$$= \begin{bmatrix} 2t^3 - 3t^2 + 1 \\ -2t^3 + 3t^2 \\ t^3 - 2t^2 + t \\ t^3 - t^2 \end{bmatrix}^T \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

$$y_1 = p_1$$

$$y_2 = p_2$$

$$m_1 = \frac{1}{2}\frac{p_2 - p_0}{x_2 - x_0}$$

$$m_2 = \frac{1}{2}\frac{p_3 - p_1}{x_3 - x_1}$$

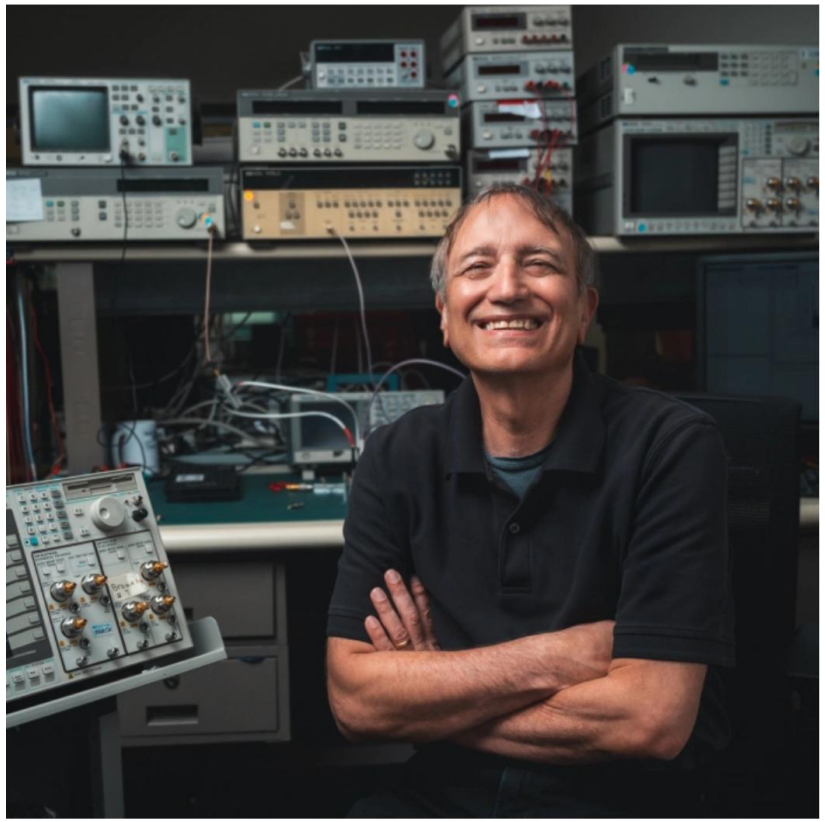


Catmull-Rom Spline

Edwin Catmull

2019 ACM Turing Award

Edwin Catmull



Pat Hanrahan

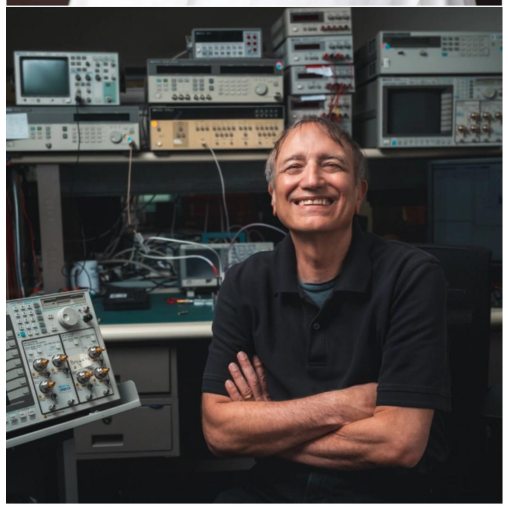
2019 ACM Turing Award

ACM named Patrick M. (Pat) Hanrahan and Edwin E. (Ed) Catmull recipients of the 2019 ACM A.M. Turing Award for fundamental contributions to 3-D computer graphics, and the revolutionary impact of these techniques on computer-generated imagery (CGI) in filmmaking and other applications. Catmull is a computer scientist and former president of Pixar and Disney Animation Studios. Hanrahan, a founding employee at Pixar, is a professor in the Computer Graphics Laboratory at Stanford University.

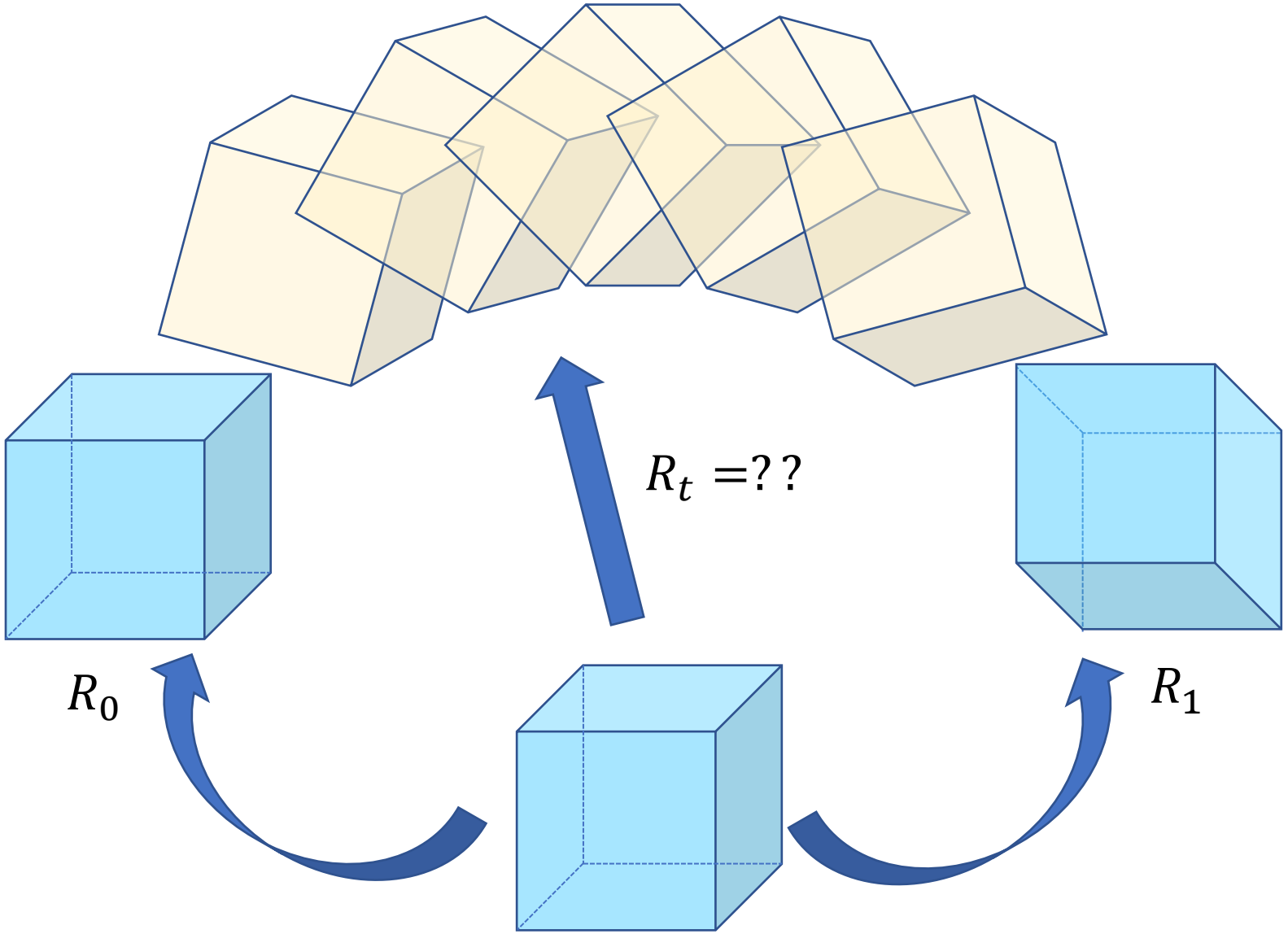
https://awards.acm.org/about/2019-turing

Edwin Catmull

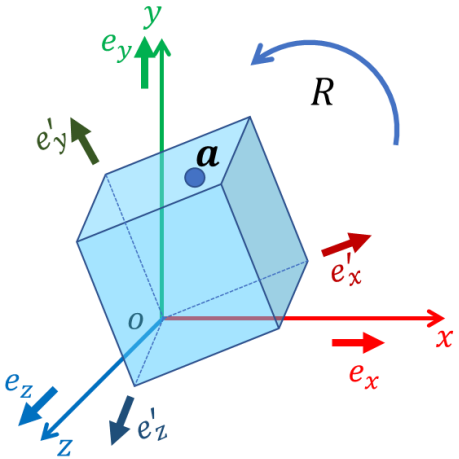
Pat Hanrahan



Interpolation of Rotations

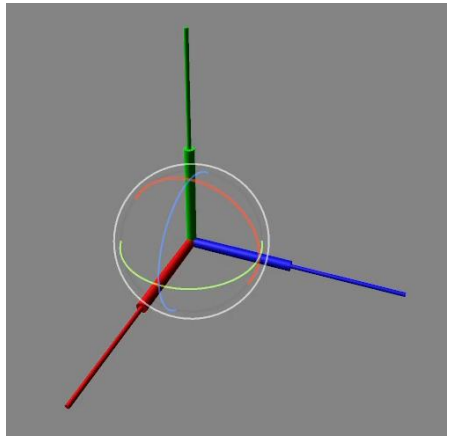


Rotation Representations



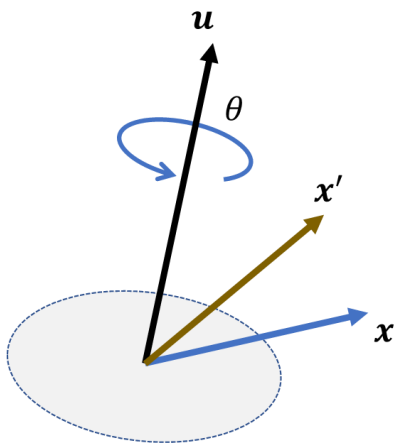
Rotation Matrix

$R_x(\alpha)R_y(\beta)R_z(\gamma)$



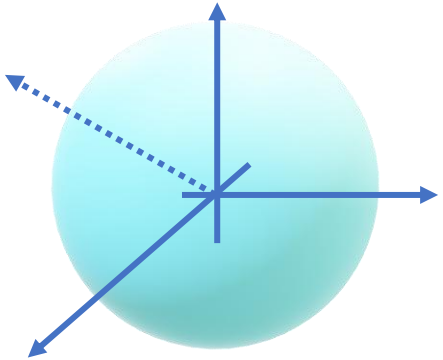
Euler Angles

$(\boldsymbol{u}, \theta)$ or $\boldsymbol{\theta}$



Axis Angles

$\boldsymbol{q} = \begin{bmatrix} w \\ \boldsymbol{v} \end{bmatrix}$



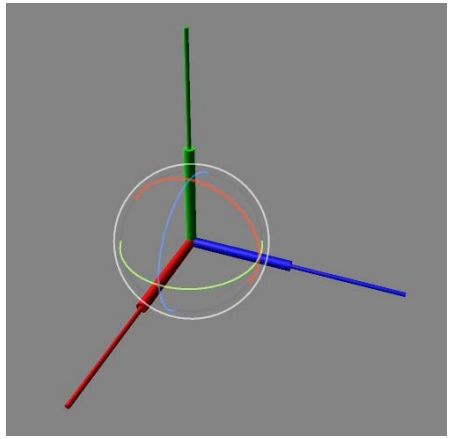
Unit Quaternions

Interpolation of Rotations

Interpolate parameters using (linear/cubic) splines

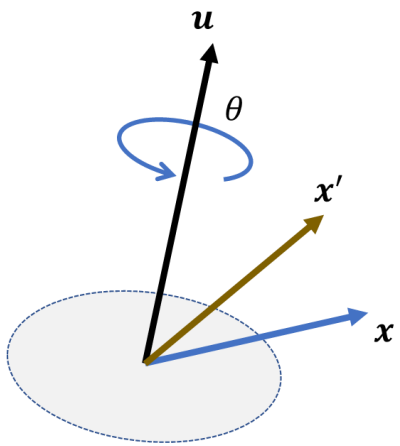
Rotation Matrix

$R_x(\alpha)R_y(\beta)R_z(\gamma)$



Euler Angles

$(\boldsymbol{u}, \theta)$ or $\boldsymbol{\theta}$



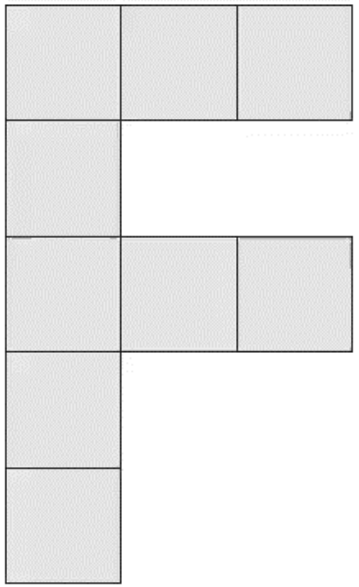
Axis Angles

$q = \begin{bmatrix} w \\ v \end{bmatrix}$

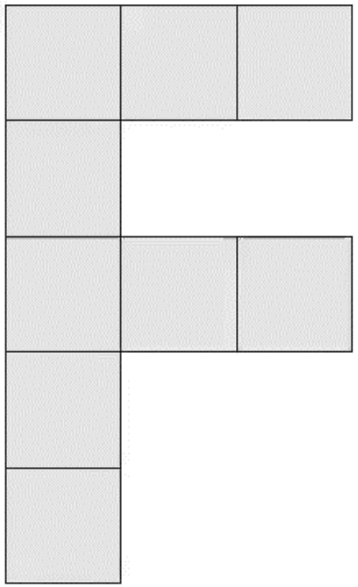
Unit Quaternions

Rotational speed is usually not constant

Interpolation of Rotations



catmull-rom euler



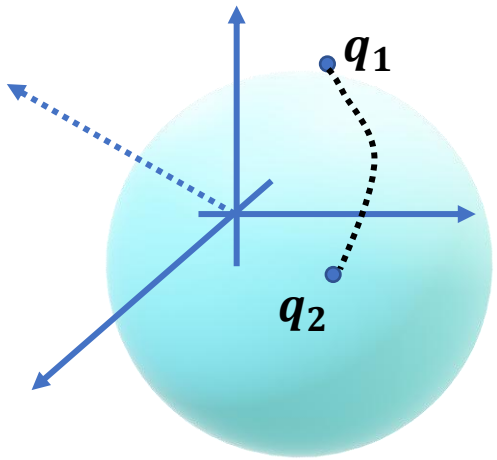
catmull-rom axis-angle

SLERP for Quaternions

$$\boldsymbol{q} = \begin{bmatrix} w \\ \boldsymbol{v} \end{bmatrix}$$

$$\boldsymbol{q}_t = \frac{\sin[(1-t)\theta]}{\sin\theta}\boldsymbol{q}_0 + \frac{\sin t\theta}{\sin\theta}\boldsymbol{q}_1$$

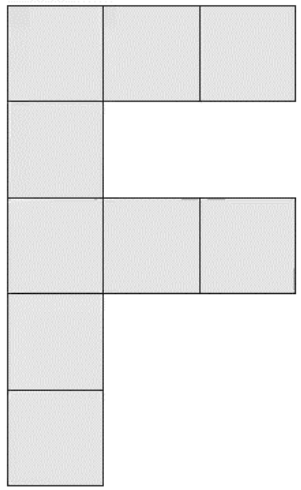
$$\cos\theta = \boldsymbol{q}_0 \cdot \boldsymbol{q}_1$$



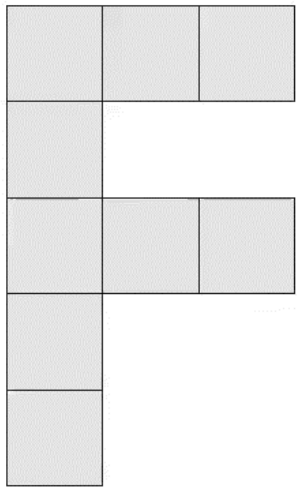
Unit Quaternions

Constant rotational speed, but only “linear” interpolation

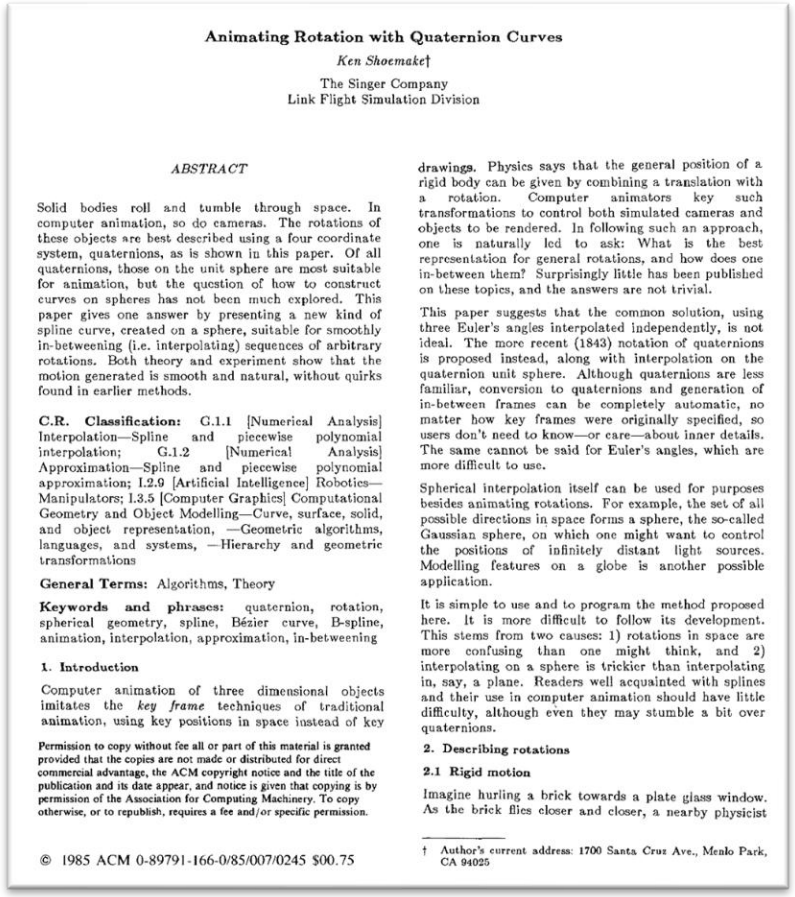
Splines for Quaternions?



piecewise Slerp



TCB interp



Animating Rotation with Quaternion Curves

Ken Shoemake†

The Singer Company
Link Flight Simulation Division

ABSTRACT

Solid bodies roll and tumble through space. In computer animation, so do cameras. The rotations of these objects are best described using a four coordinate system, quaternions, as is shown in this paper. Of all quaternions, those on the unit sphere are most suitable for animation, but the question of how to construct curves on spheres has not been much explored. This paper gives one answer by presenting a new kind of spline curve, created on a sphere, suitable for smoothly in-betweening (i.e. interpolating) sequences of arbitrary rotations. Both theory and experiment show that the motion generated is smooth and natural, without quirks found in earlier methods.

C.R. Classification: G.1.1 [Numerical Analysis] Interpolation—Spline and piecewise polynomial interpolation; G.1.2 [Numerical Analysis] Approximation—Spline and piecewise polynomial approximation; I.2.9 [Artificial Intelligence] Robotics—Manipulators; I.3.5 [Computer Graphics] Computational Geometry and Object Modelling—Curve, surface, solid, and object representation, —Geometric algorithms, languages, and systems, —Hierarchy and geometric transformations

General Terms: Algorithms, Theory

Keywords and phrases: quaternion, rotation, spherical geometry, spline, Bézier curve, B-spline, animation, interpolation, approximation, in-betweening

1. Introduction

Computer animation of three dimensional objects imitates the *key frame* techniques of traditional animation, using key positions in space instead of key drawings. Physics says that the general position of a rigid body can be given by combining a translation with a rotation. Computer animators key such transformations to control both simulated cameras and objects to be rendered. In following such an approach, one is naturally led to ask: What is the best representation for general rotations, and how does one in-between them? Surprisingly little has been published on these topics, and the answers are not trivial.

This paper suggests that the common solution, using three Euler's angles interpolated independently, is not ideal. The more recent (1843) notation of quaternions is proposed instead, along with interpolation on the quaternion unit sphere. Although quaternions are less familiar, conversion to quaternions and generation of in-between frames can be completely automatic, no matter how key frames were originally specified, so users don't need to know—or care—about inner details. The same cannot be said for Euler's angles, which are more difficult to use.

Spherical interpolation itself can be used for purposes besides animating rotations. For example, the set of all possible directions in space forms a sphere, the so-called Gaussian sphere, on which one might want to control the positions of infinitely distant light sources. Modelling features on a globe is another possible application.

It is simple to use and to program the method proposed here. It is more difficult to follow its development. This stems from two causes: 1) rotations in space are more confusing than one might think, and 2) interpolating on a sphere is trickier than interpolating in, say, a plane. Readers well acquainted with splines and their use in computer animation should have little difficulty, although even they may stumble a bit over quaternions.

2. Describing rotations

2.1 Rigid motion

Imagine hurling a brick towards a plate glass window. As the brick flies closer and closer, a nearby physicist

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

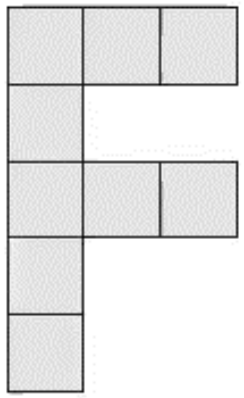
© 1985 ACM 0-89791-166-0/85/007/0245 $00.75

† Author's current address: 1700 Santa Cruz Ave., Menlo Park, CA 94025

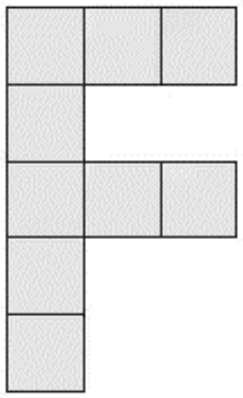
Ken Shoemake. 1985
Animating rotation with quaternion curves.
SIGGRAPH Computer Graphics,

https://splines.readthedocs.io/en/latest/rotation/kochanek-bartels.html
About TCP interpolation: https://dl.acm.org/doi/10.1145/800031.808575

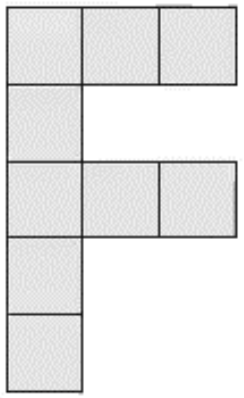
Splines for Quaternions?



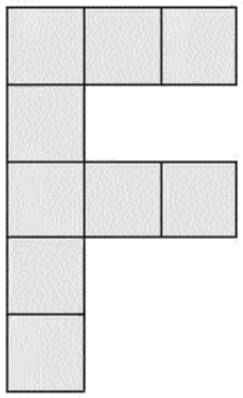
catmull-rom euler



catmull-rom axis-angle

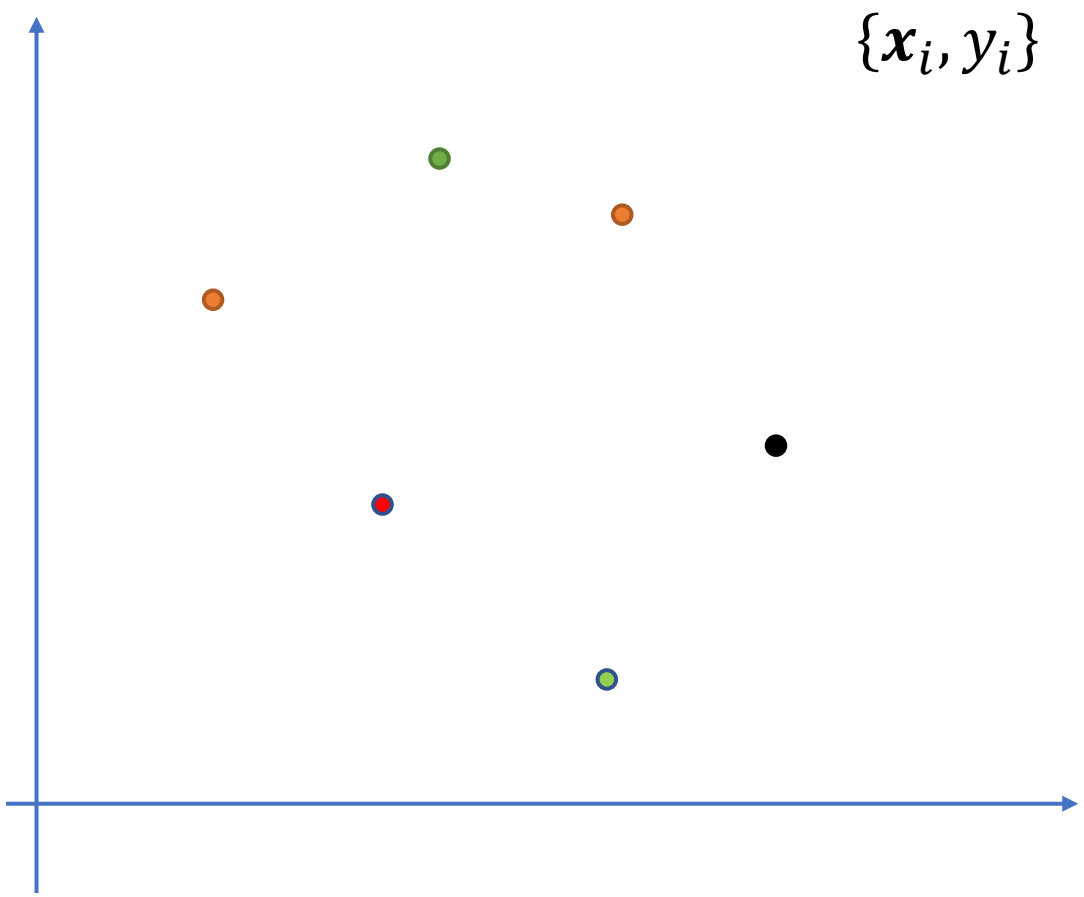


piecewise Slerp

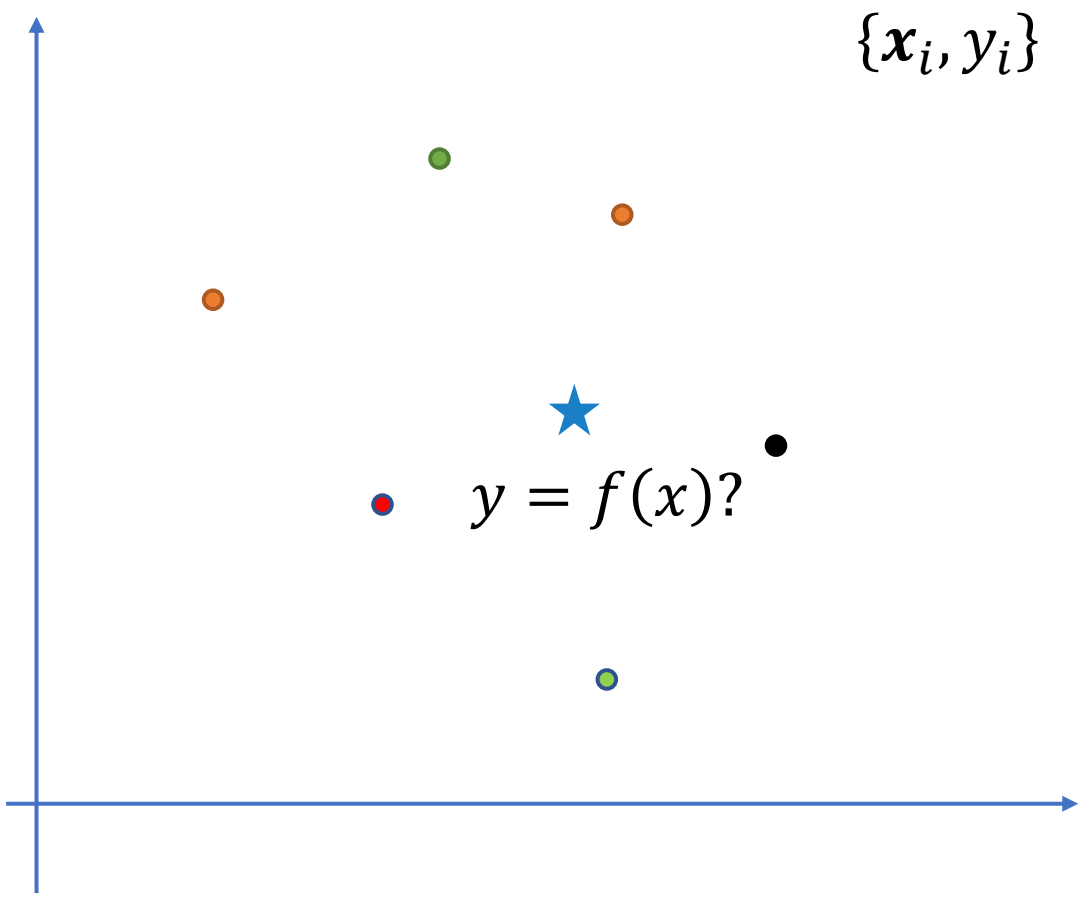


TCB interp

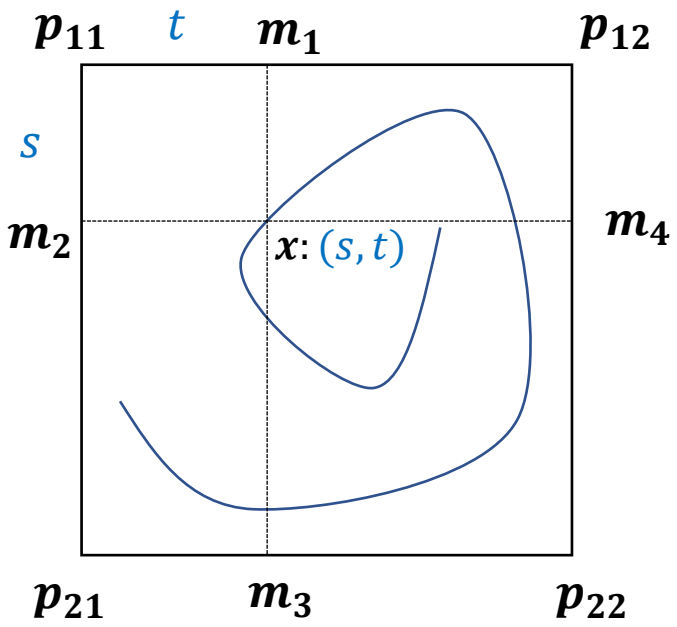
Interpolation in a 2D space?



Interpolation in a 2D space?

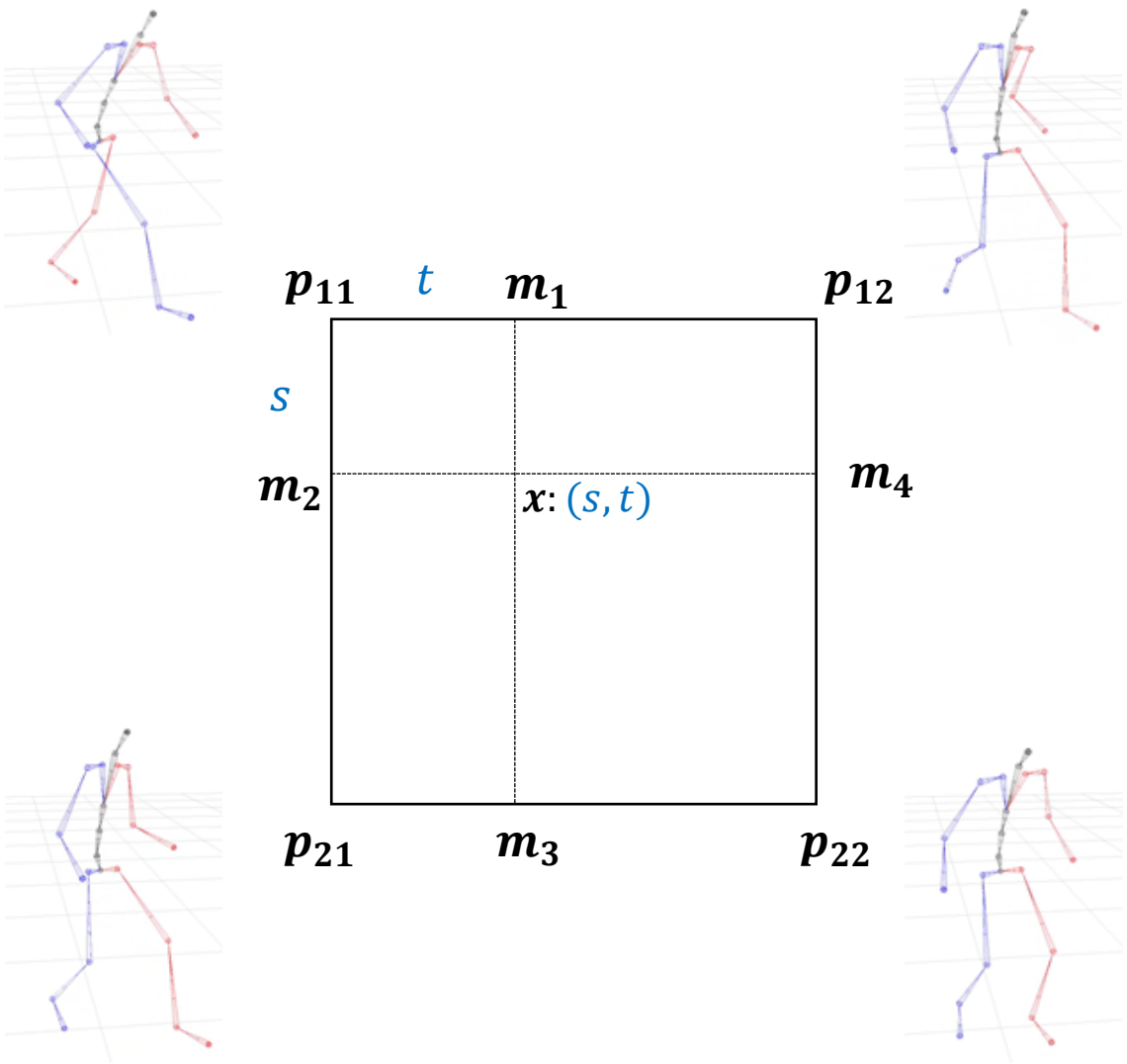


Bilinear Interpolation



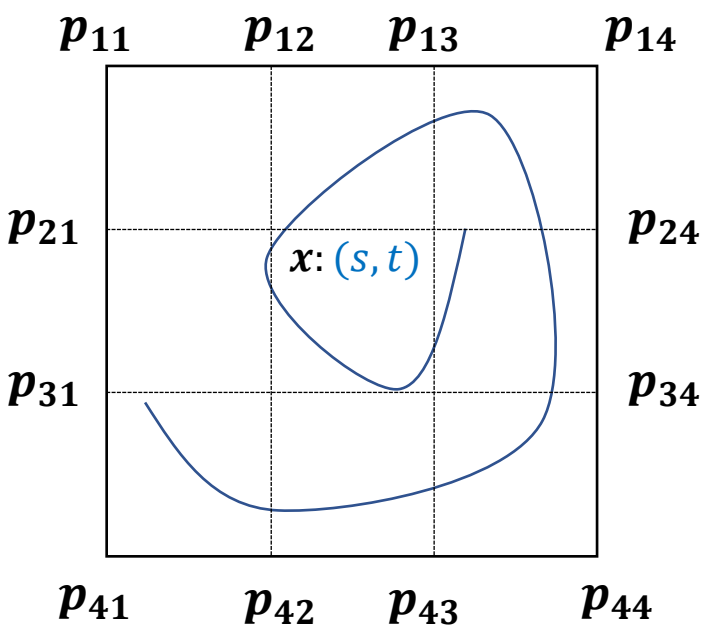
$$\begin{aligned}\boldsymbol{x} &= (1-t)\boldsymbol{m}_2 + t\boldsymbol{m}_4 \\ &= (1-t)\big((1-s)\boldsymbol{p}_{11} + s\boldsymbol{p}_{21}\big) \\ &+ \quad t\ \big((1-s)\boldsymbol{p}_{12} + s\boldsymbol{p}_{22}\big)\end{aligned}$$

Bilinear Interpolation



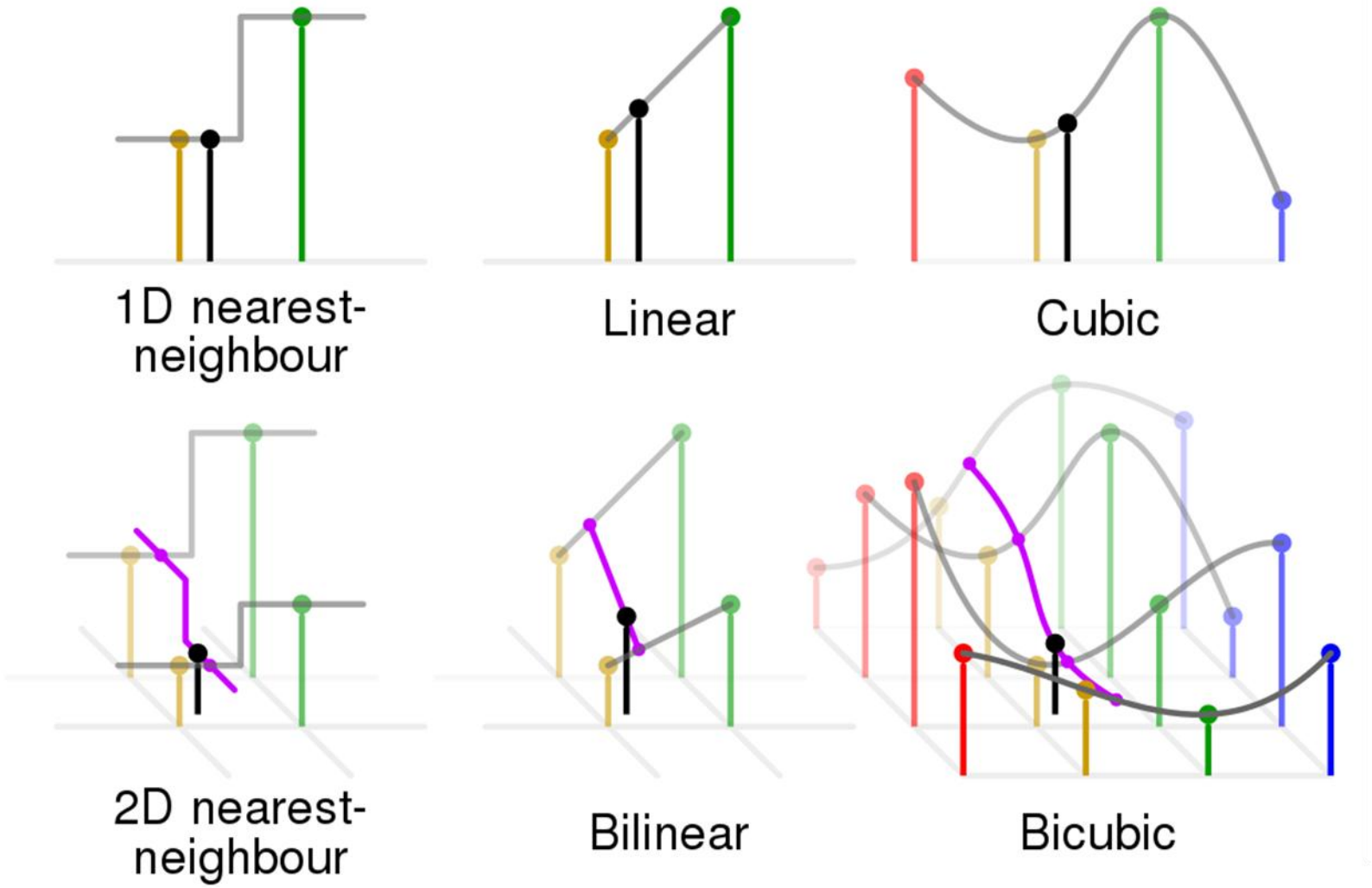
$$\begin{aligned}\boldsymbol{x} &= (1-t)\boldsymbol{m}_2 + t\boldsymbol{m}_4 \\ &= (1-t)\big((1-s)\boldsymbol{p}_{11} + s\boldsymbol{p}_{21}\big) \\ &+ \quad t\ \big((1-s)\boldsymbol{p}_{12} + s\boldsymbol{p}_{22}\big)\end{aligned}$$

Bicubic Interpolation?

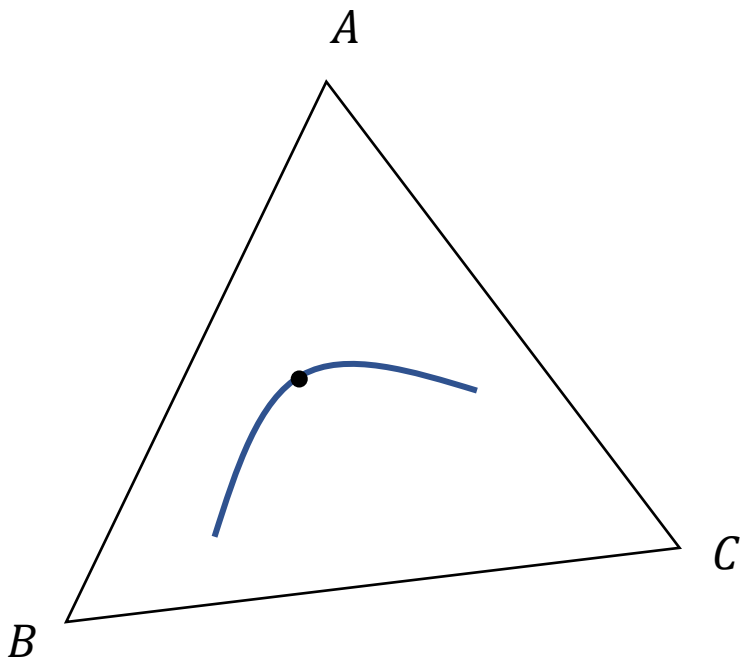


$x = ?$

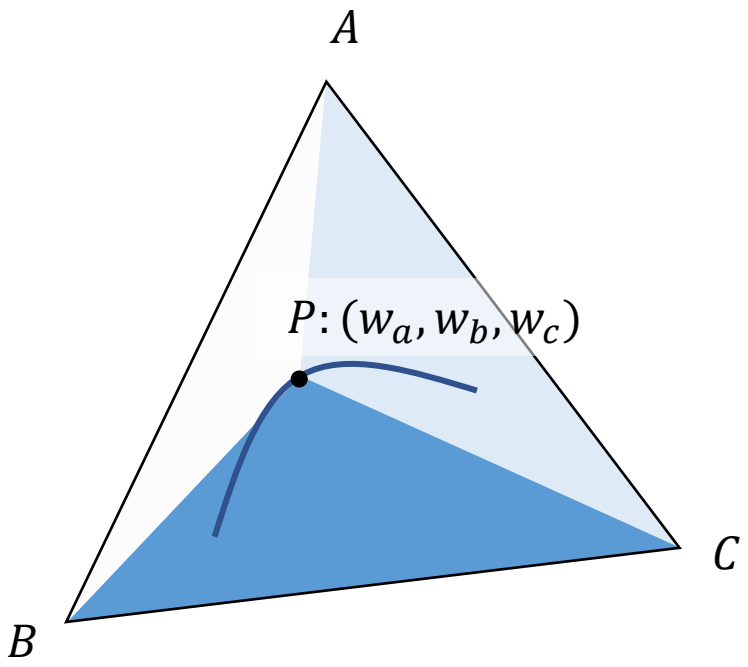
Bilinear Interpolation



Barycentric Interpolation



Barycentric Interpolation



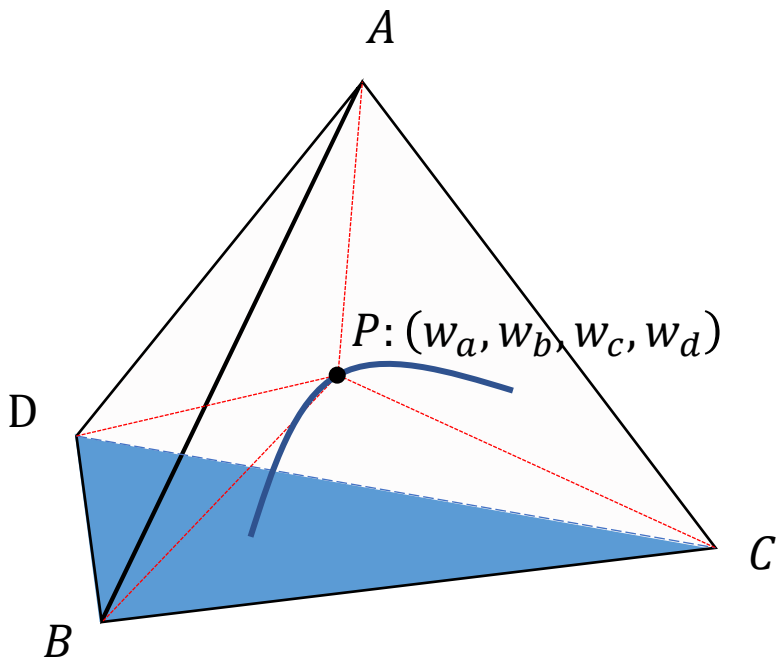
$$P = w_a A + w_b B + w_c C$$

$$w_a = \frac{\Delta PBC}{\Delta ABC}$$

$$w_b = \frac{\Delta PCA}{\Delta ABC}$$

$$w_c = \frac{\Delta PAC}{\Delta ABC}$$

Barycentric Interpolation



$$P = w_a A + w_b B + w_c C + w_d D$$

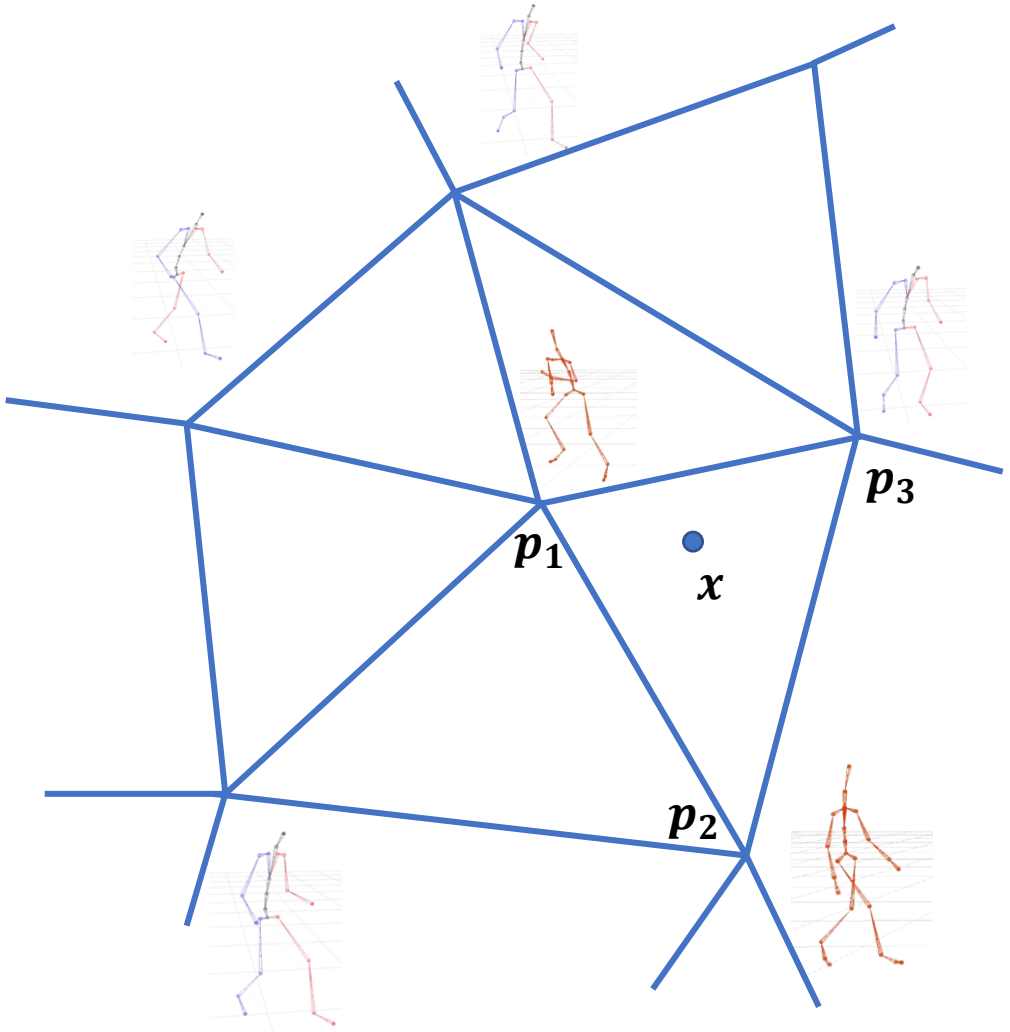
$$w_a = \frac{\Delta PBCD}{\Delta ABCD}$$

$$w_b = \frac{\Delta PCDA}{\Delta ABCD}$$

$$w_c = \frac{\Delta PDAB}{\Delta ABCD}$$

$$w_d = \frac{\Delta PABC}{\Delta ABCD}$$

Blend Space

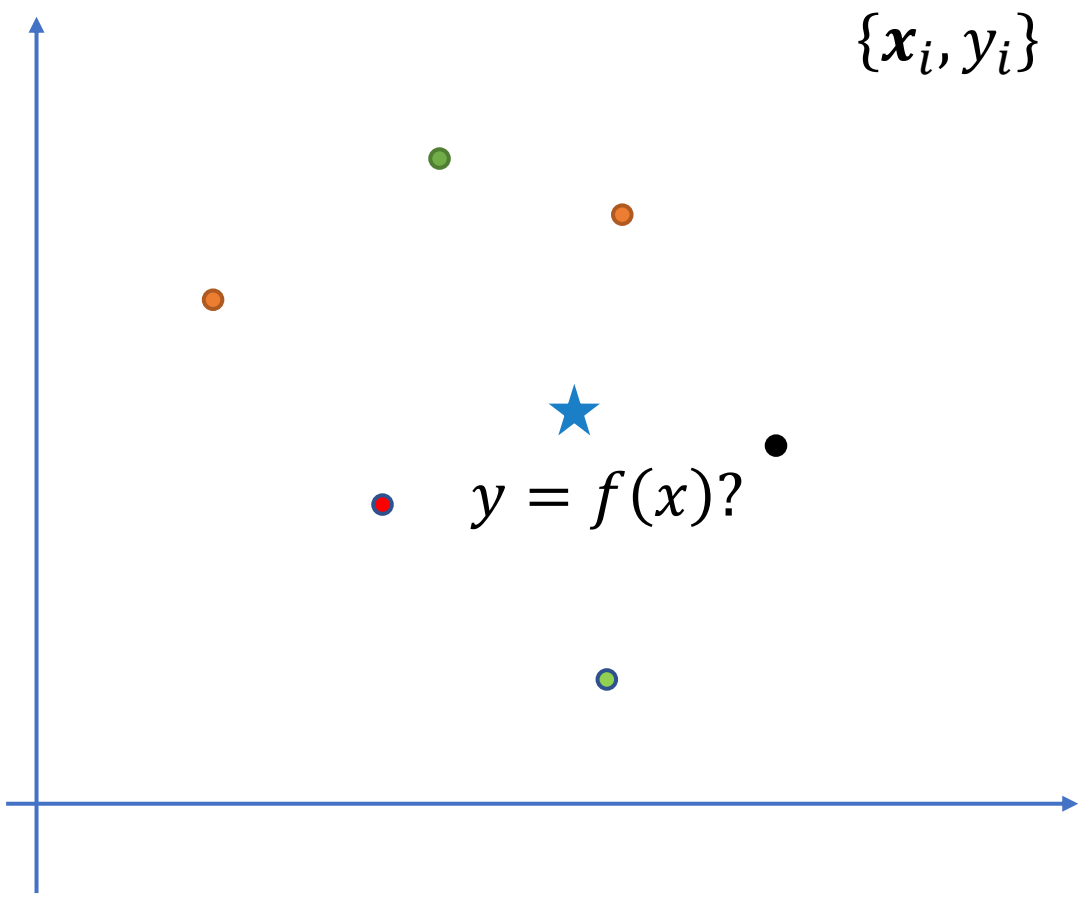


$$x = w_a p_1 + w_b p_2 + w_c p_3$$

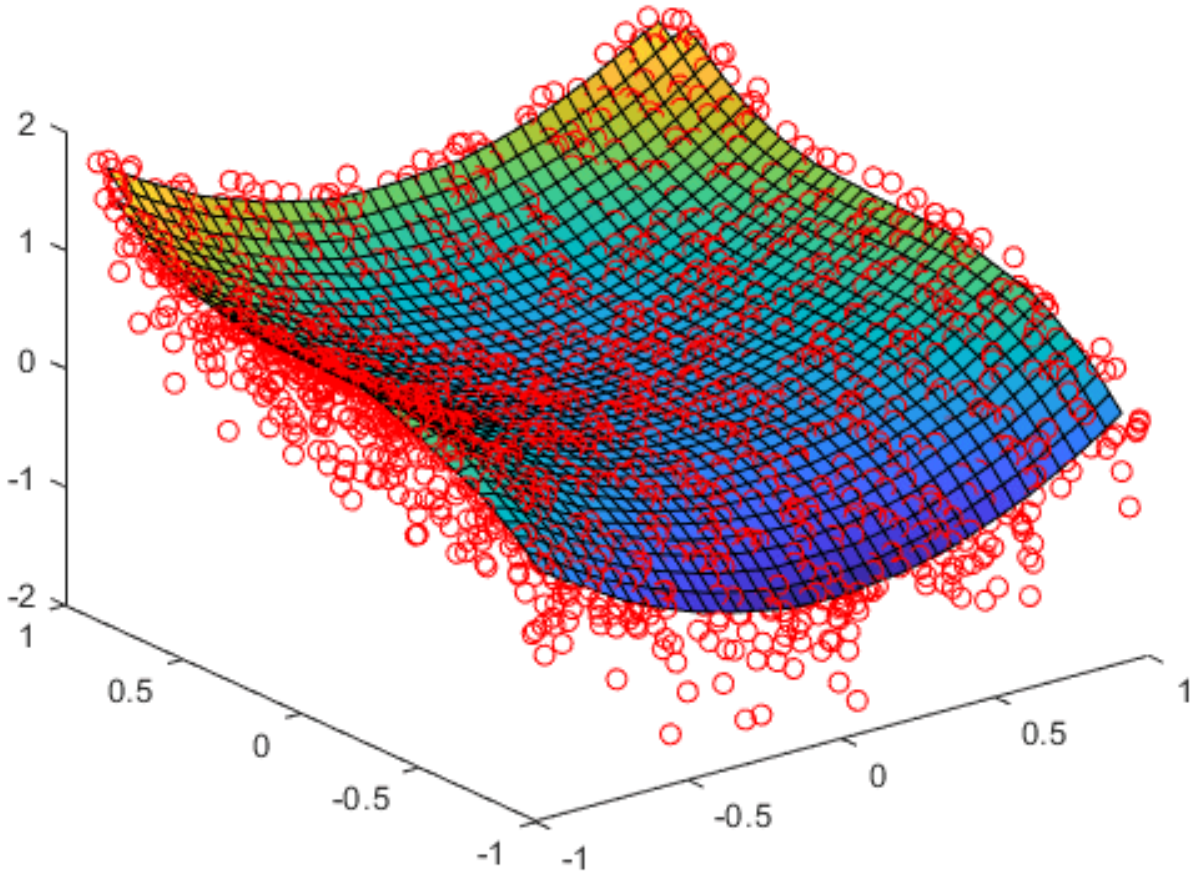
2D Blendspace

Using A 2D Blendspace - #8 Unreal Engine 4 Animation Essentials Tutorial Series | https://www.youtube.com/watch?v=Bn16sBLxjCQ

Scattered Data Interpolation 散点数据插值



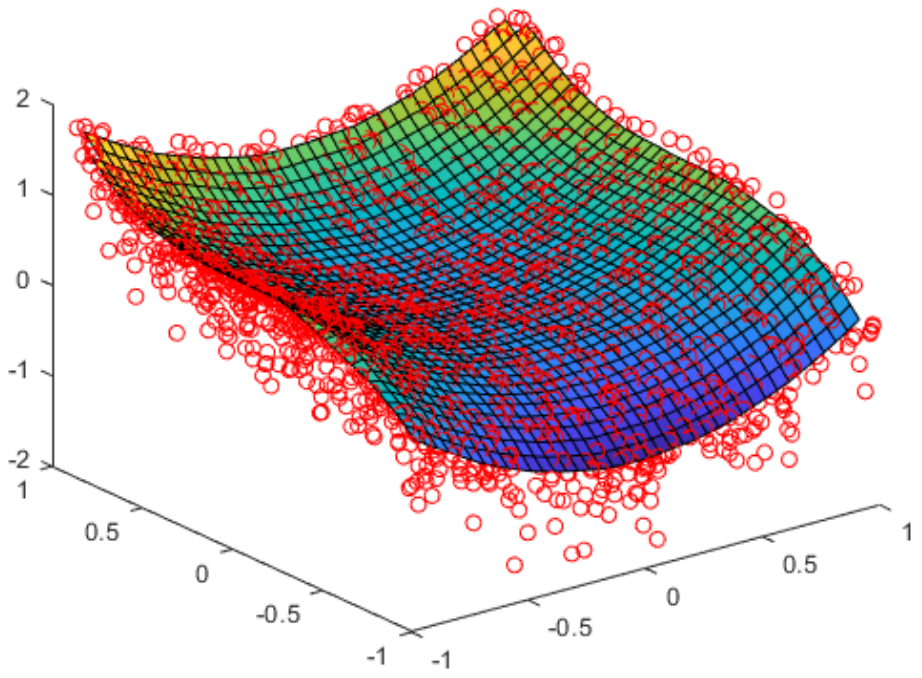
Scattered Data Interpolation



https://www.mathworks.com/help/matlab/ref/griddata.html

Scattered Data Interpolation

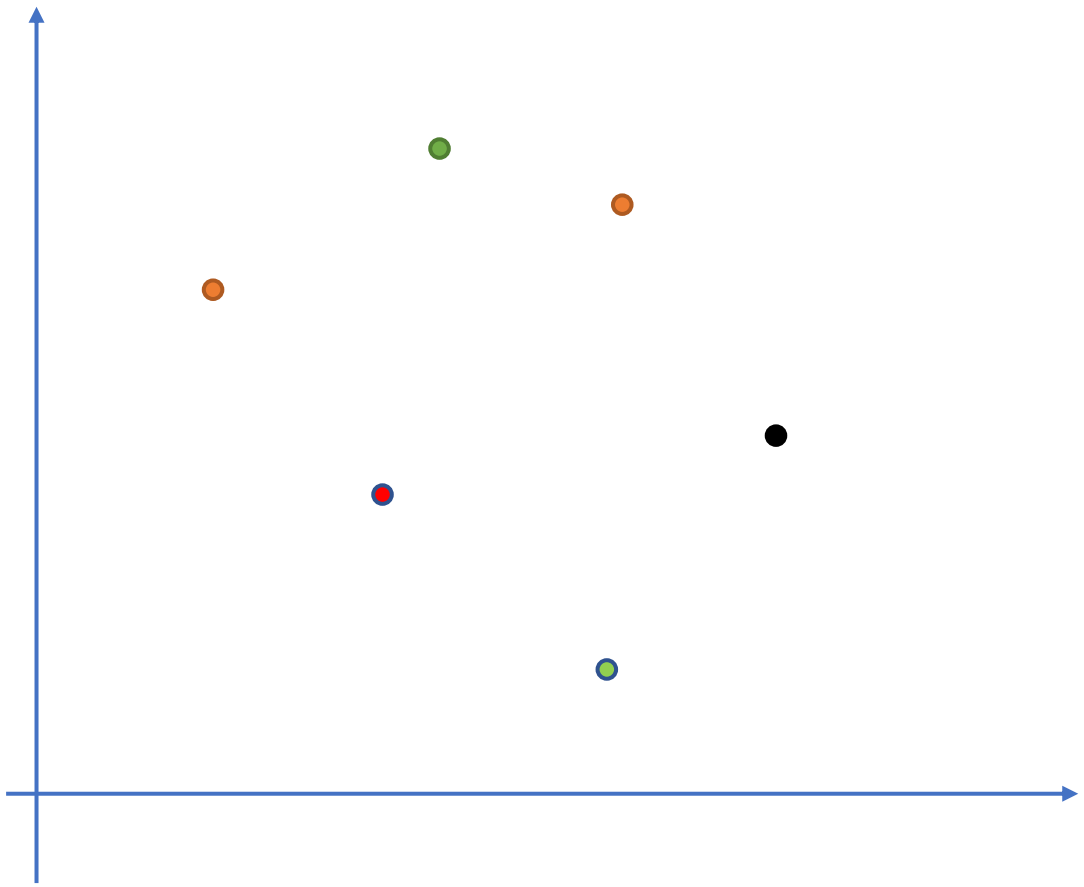
- Linear
  - Least squares
- Splines
- Inverse distance weighting
- Gaussian process
- Radial Basis Function
- ……



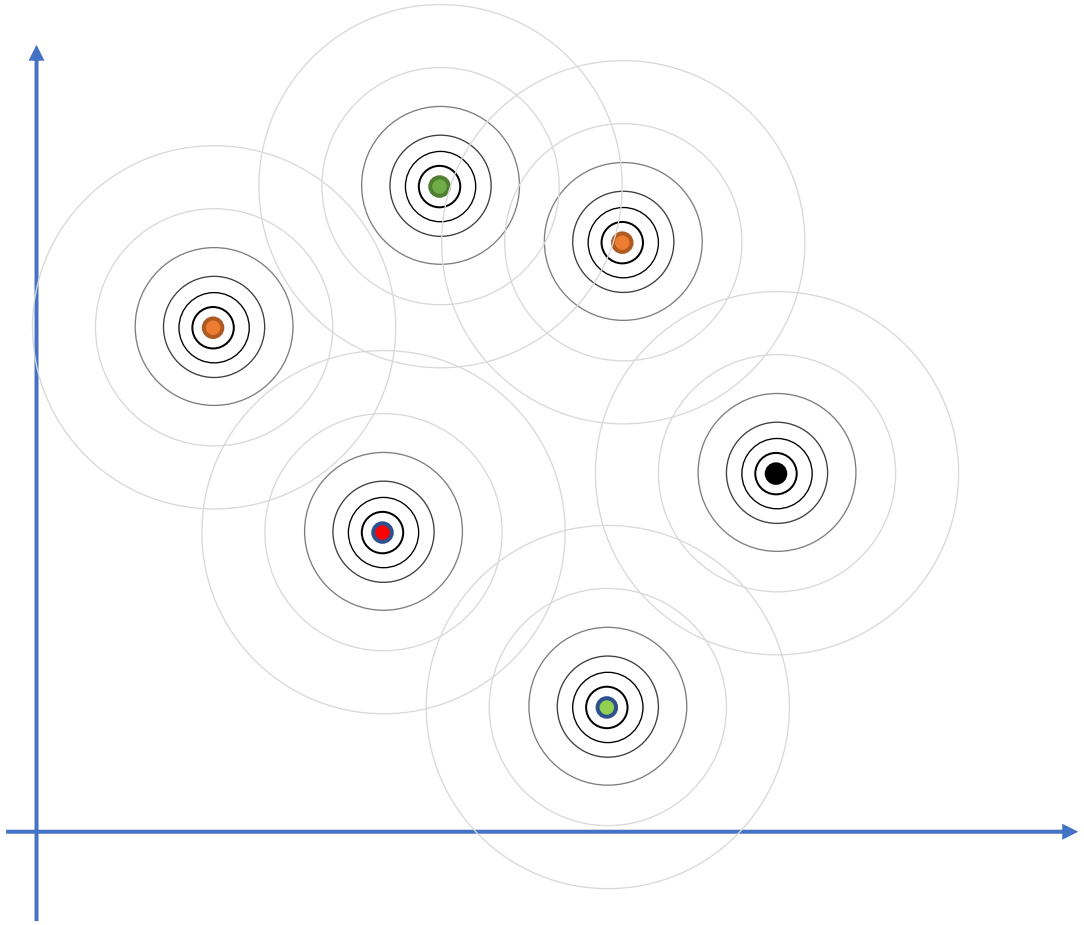
https://www.mathworks.com/help/matlab/ref/griddata.html

Radial Basis Function (RBF) Interpolation

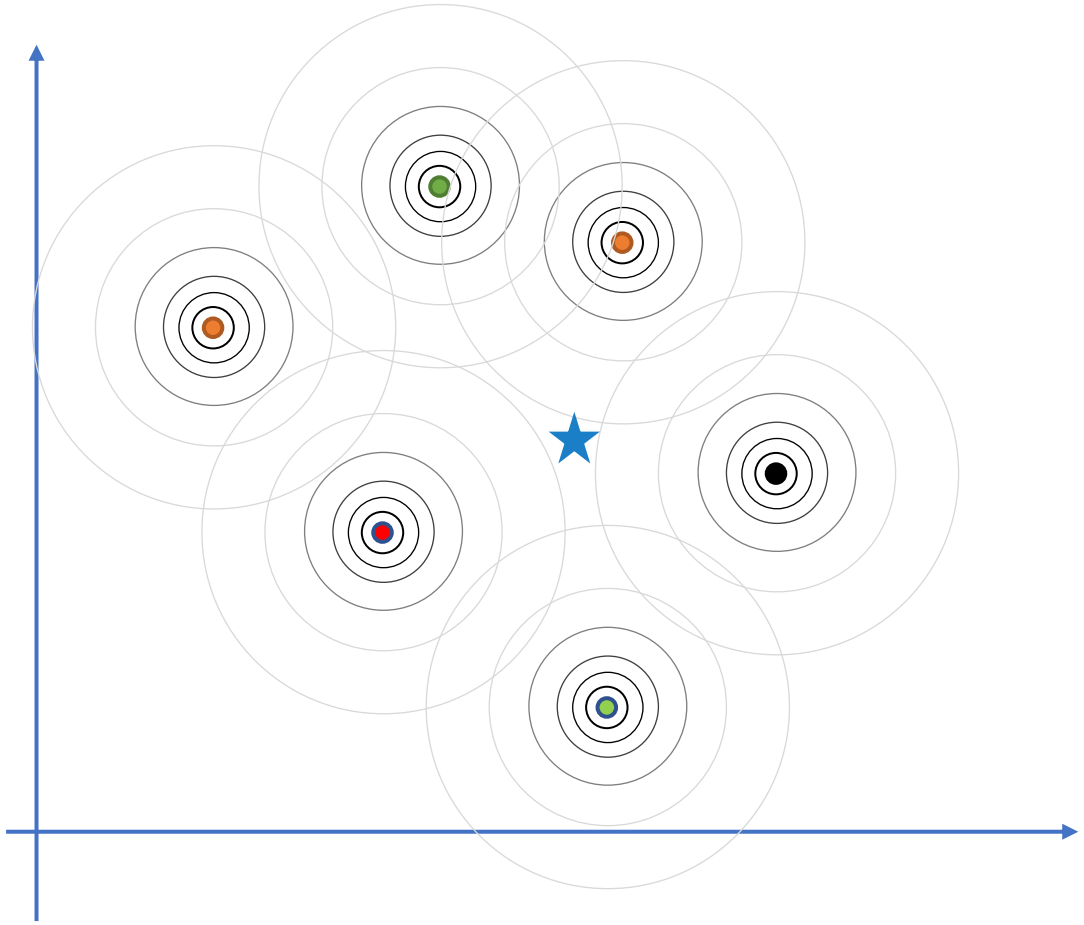
径向基函数插值



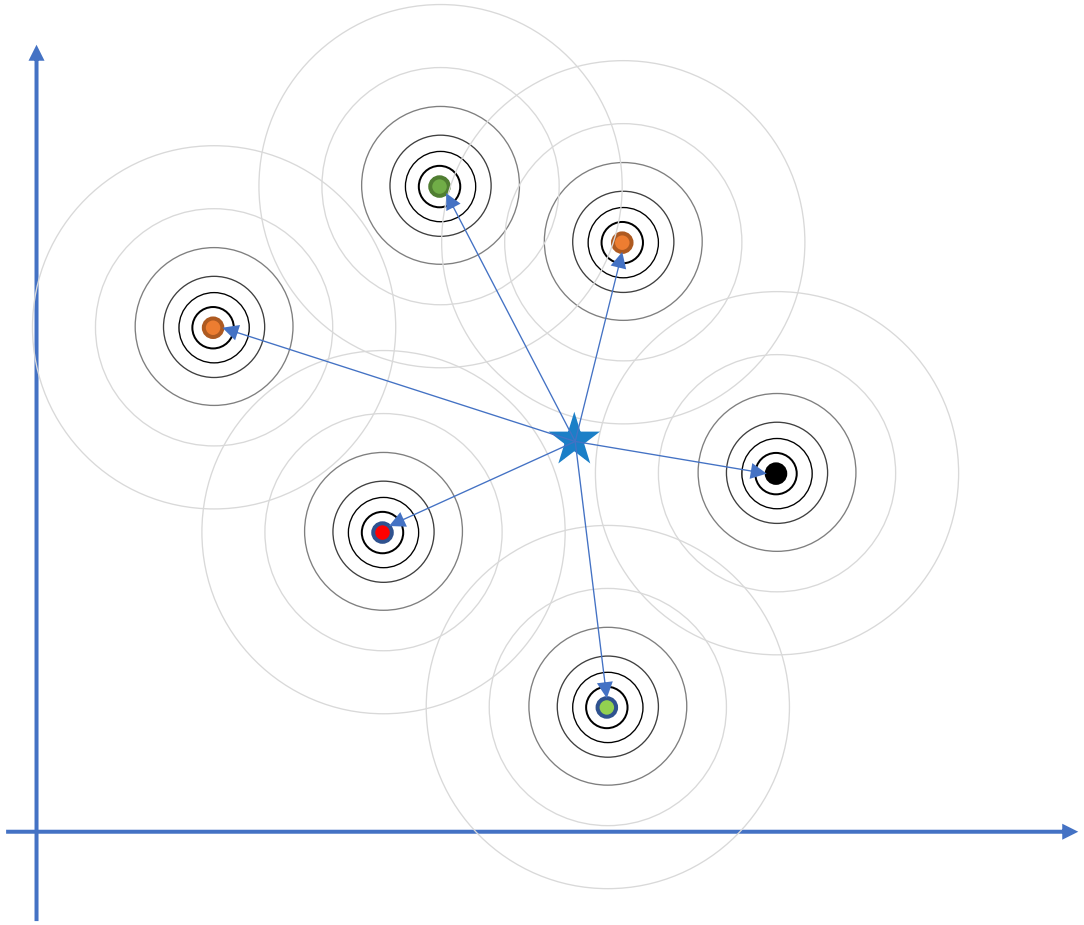
Radial Basis Function (RBF) Interpolation



Radial Basis Function (RBF) Interpolation

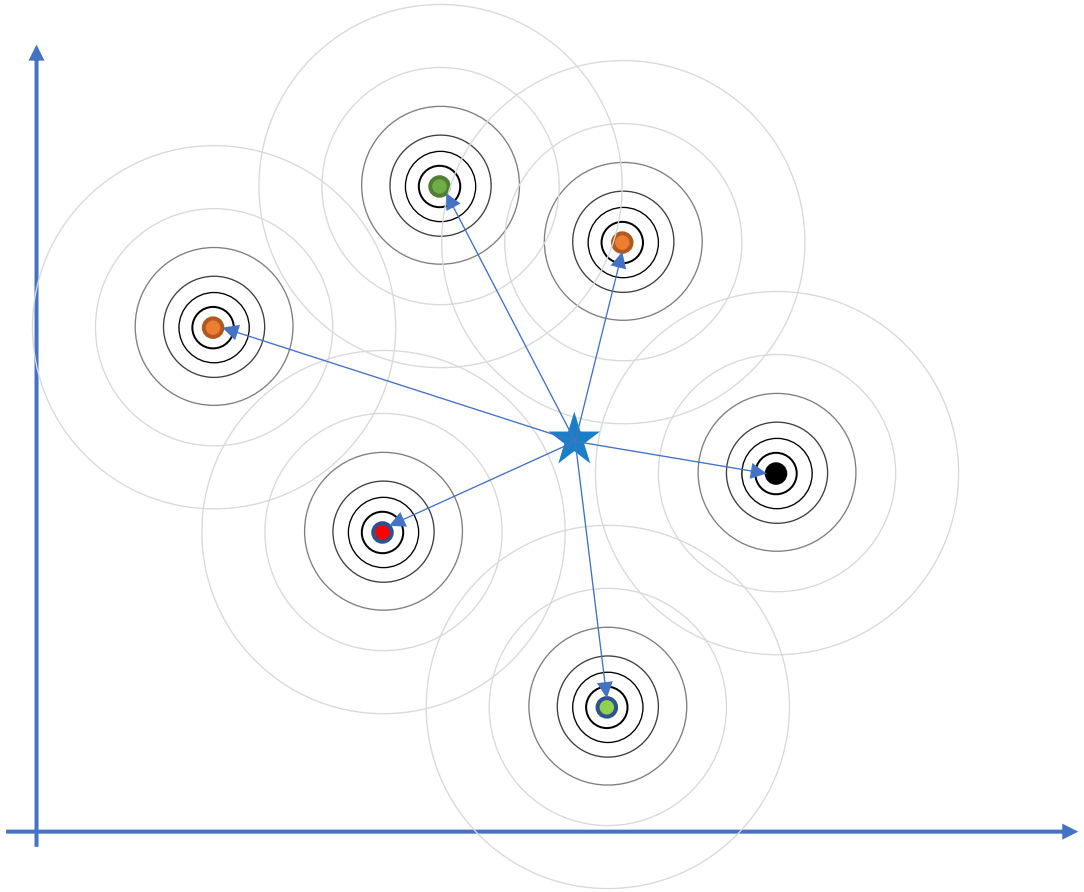


Radial Basis Function (RBF) Interpolation



Radial Basis Function (RBF) Interpolation

$$y = \sum_{i=1}^{K} w_i \varphi(\|\boldsymbol{x} - \boldsymbol{x}_i\|)$$



Radial Basis Function (RBF) Interpolation

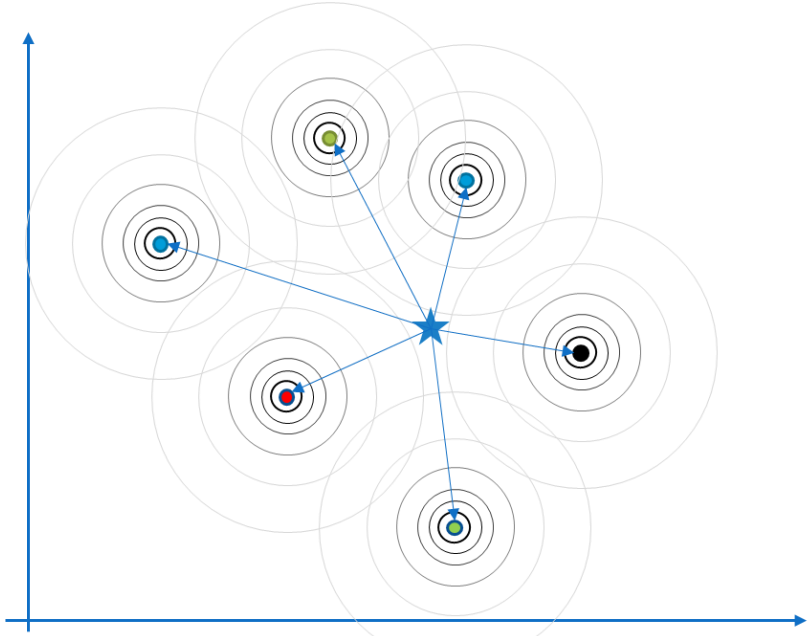
$$y = \sum_{i=1}^{K} w_i \varphi(\|\boldsymbol{x} - \boldsymbol{x}_i\|)$$

How to compute w_i ?

Radial Basis Function (RBF) Interpolation

$$y = \sum_{i=1}^{K} w_i \varphi(\|\boldsymbol{x} - \boldsymbol{x}_i\|)$$

How to compute w_i ? We need $f(\boldsymbol{x}_i) = y_i$



$$\begin{bmatrix} R_{1,1} & R_{1,2} & \cdots & R_{1,K} \\ R_{2,1} & R_{2,2} & & \vdots \\ \vdots & & \ddots & \vdots \\ R_{K,1} & \cdots & \cdots & R_{K,K} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_K \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_K \end{bmatrix}$$

$$R_{i,j} = \varphi(\|\boldsymbol{x}_i - \boldsymbol{x}_j\|)$$

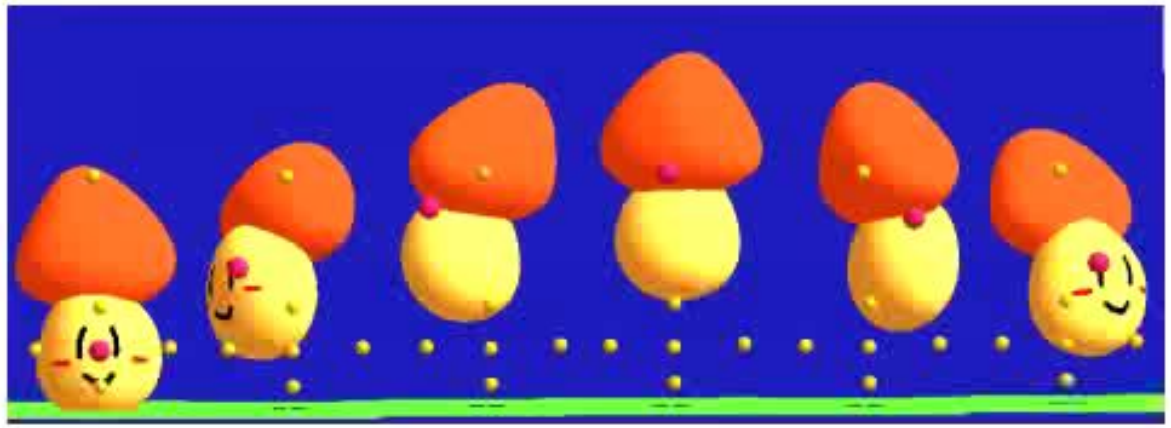
Radial Basis Function (RBF)

$$y = \sum_{i=1}^{K} w_i \varphi(\|\boldsymbol{x} - \boldsymbol{x}_i\|)$$

- Gaussian: $\varphi(r) = e^{-(r/c)^2}$
- Inverse multiquadric: $\varphi(r) = \frac{1}{\sqrt{r^2+c^2}}$
- Thin plate spline: $\varphi(r) = r^2 \log r$
- Polyharmonic splines: $\varphi(r) = \begin{cases} r^k, & k = 2n+1 \\ r^k \log r, & k = 2n \end{cases}$

RBF Interpolation

Spatial Keyframing for Performance-driven Animation



Spatial Keyframing for Performance-driven Animation

T. Igarashi [1,3], T. Moscovich [2], and J. F. Hughes [2]

[1] The University of Tokyo [2] Brown University [3] PRESTO, JST

Abstract

This paper introduces spatial keyframing, a technique for performance-driven character animation. In traditional temporal keyframing, key poses are defined at specific points in time: i.e., we define a map from a set of key times to the configuration space of the character and then extend this map to the entire timeline by interpolation. By contrast, in spatial keyframing key poses are defined at specific key positions in a 3D space where the character lives; the mapping from the 3D space to the configuration space is again defined by interpolation. The user controls a character by adjusting the position of a control cursor in the 3D space; the pose of the character is given as a blend of nearby key poses. The user thus can make expressive motion in real time and the resulting motion can be recorded and interpreted as an animation sequence. Although similar ideas are present in previous systems, our system is unique in that the designer can quickly design a new set of keyframes from scratch, and make an animation without motion capture data or special input devices. Our technique is especially useful for imaginary characters other than human figures because we do not rely on motion-capture data. We also introduce several applications of the basic idea and give examples showing the expressiveness of the approach.

1. Introduction

The most popular approach to character animation is keyframing, where the designer manually specifies the pose of a character as a discrete set of frames (keyframes) and the computer synthesizes the poses in the remaining frames. However, novice users have difficulty in creating fluid motion using this approach and it is very labor-intensive work. Other approaches such as motion capture and physically based simulation are available, but they are expensive to use and are not suitable for designing expressive imaginary motions. Furthermore, motion capture is mainly designed for human figures and is not directly applicable to imaginary characters.

We describe here a method that lets novice users create lively animations for arbitrary 3D characters quickly and easily using a standard input device such as a mouse. The basic idea is to directly record the user's performance or actions, that is, the user's direct manipulation of the character. We believe that this is much faster and more intuitive than traditional temporal keyframing, because the user need not mentally translate static keyframes to temporal motion during design. In performance-driven animation, what you perform and see on the screen during recording is what you obtain as the final result.

Figure 1: *Spatial keyframing with six key poses (top) and an example animation sequence using it (bottom). The user associates each pose with a location in space (yellow markers) in a preparation phase. During performance, the user moves the control cursor (red sphere) and the system synthesizes an animation sequence by blending nearby poses.*

Igarashi et al. 2005. Spatial Keyframing for Performance-driven Animation. *SCA 2005*

Outline

- Keyframe Animation
  - Interpolation and splines

Questions?